

**CLIMATE-SMART COTTON YIELD
PREDICTION: A COMPARATIVE STUDY OF
RANDOM FOREST AND LIGHTGBM**

*A project Report Submitted Towards the Partial
Requirement for the Award of the Degree*

**BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

Satuluri Mohammad Sakhib	(22471A0539)
Gunji Venkata Ganesh	(22471A0525)
Desireddy Charan Venkata Siva Nagi Reddy	(22471A0516)

Under the esteemed guidance of
D VENKATA REDDY, M. Tech,(Ph.D.)
Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NARASARAOPETA ENGINEERING COLLEGE: NARASARAOPETA
(AUTONOMOUS)

**Accredited by NAAC with A++ Grade and NBA under Tyre -1
and an ISO 9001:2015 Certified**

**Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK,
Kakinada, Narasaraopeta-522601, palnadu(Dt), Andhra Pradesh,
India, 2025-2026**

NARASARAOPETA ENGINEERING COLLEGE

(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND

ENGINEERING



CERTIFICATE

This is to certify that the project that is entitled with the name **“Climate-Smart Cotton Yield Prediction: A Comparative Study of Random Forest and LightGBM”** is a Bonafide work done by the team in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of COMPUTER SCIENCE AND ENGINEERING during 2025-2026

PROJECT GUIDE

D. Venkata Reddy, M.Tech., (Ph.D.)

PROJECT CORDINATOR

Dr. Sireesha Moturi, , M.Tech., Ph.D.

HEAD OF THE DEPARTMENT

Dr. S. N. Tirumala Rao, M.Tech., Ph.D.

EXTERNAL EXAMINER

DECLARATION

We declare that this project work titled “**Climate-Smart Cotton Yield Prediction: A Comparative Study of Random Forest and LightGBM**” is composed by ourselves that the work contains here is our own except where explicitly stated otherwise in the text and that this work has been not submitted for any other degree or professional qualification except as specified.

By

Satuluri Mohammadh Sakhib

(22471A0539)

Gunji Venkata Ganesh

(22471A0525)

Desireddy Charan Venkata Siva

Nagi Reddy

(22471A0516)

ACKNOWLEDGEMENT

We wish to express my thanks to carious personalities who are responsible for the completion of the project. We are extremely thankful to our beloved chairman sri **M. V. Koteswara Rao**, B.Sc., who took keen interest in us in every effort throughout thiscourse. We owe out sincere gratitude to our beloved principal **Dr. S. Venkateswarlu**, Ph.D., for showing his kind attention and valuable guidance throughout the course.

We express our deep felt gratitude towards **Dr. S. N. Tirumala Rao**, M.Tech., Ph.D., HOD of CSE department and also to our guide **D. Venkata Reddy**, M.Tech., (Ph.D.,) of CSE department whose valuable guidance and unstinting encouragement enable us to accomplish our project successfully in time.

We extend our sincere thanks towards **Dr. Sireesha Moturi**, M.Tech., ph.D., Associate professor & Project coordinator of the project for extending her encouragement. Their profound knowledge and willingness have been a constant source of inspiration for us throughout this project work.

We extend our sincere thanks to all other teaching and non-teaching staff to department for their cooperation and encouragement during our B.Tech degree.

We have no words to acknowledge the warm affection, constant inspiration and encouragement that we received from our parents.

We affectionately acknowledge the encouragement received from our friends and those who involved in giving valuable suggestions had clarifying out doubts which had really helped us in successfully completing our project.



INSTITUTE VISION AND MISSION

INSTITUTION VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community,

INSTITUTION MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research

M2: Build a passionate and a determined team of faculty with student centric teaching, imbining experiential, innovative skills

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To become a center of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertise in modern software tools and technologies to cater to the real time requirements of the Industry.

M3: Inculcate team work and lifelong learning among students with a sense of societal and ethical responsibilities.

Program Specific Outcomes (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era in Computer Science and Engineering

PSO3: Promote novel applications that meet the needs of entrepreneur, environmental and social issues.

Program Educational Objectives (PEO's)

The graduates of the programme are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to academia, industry and society.

PEO3: Work with ethical and moral values in the multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

Program Outcomes

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Project Course Outcomes (CO'S):

CO421.1: Analyse the System of Examinations and identify the problem.

CO421.2: Identify and classify the requirements.

CO421.3: Review the Related Literature

CO421.4: Design and Modularize the project

CO421.5: Construct, Integrate, Test and Implement the Project.

CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1		✓										✓		
C421.2	✓		✓		✓							✓		
C421.3				✓		✓	✓	✓				✓		
C421.4			✓			✓	✓	✓				✓	✓	
C421.5					✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6									✓	✓	✓	✓	✓	

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
C421.1	2	3											2		
C421.2			2		3								2		
C421.3				2		2	3	3					2		
C421.4			2			1	1	2					3	2	
C421.5					3	3	3	2	3	2	2	1	3	2	1
C421.6									3	2	1		2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's:

1. Low level

2. Medium level

3. High level

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are applied in this project	Description of the device/work	Attained po
C2204.2, C22L3.2	Gathering climatic and agronomic data (heat units, soil type, cultivar, fertilizer application) and defining the cotton yield prediction problem	PO1, PO2, PO8
CC421.1, C2204.3, C22L3.2	Each predictive feature is critically analyzed, preprocessing pipelines are identified (outlier removal, categorical encoding, dataset balancing)	PO2, PO4, PO6
CC421.2, C2204.2, C22L3.3	Logical design of machine learning workflow using Random Forest and LightGBM, involving teamwork and modularization	PO3, PO5, PO8, PO9
CC421.3, C2204.3, C22L3.2	Model training, testing, integration, and evaluation using statistical metrics (RMSE, R^2 , NSE, Accuracy)	PO1, PO4, PO5
CC421.4, C2204.4, C22L3.2	Documentation of methodology, experimental results, and comparative analysis collaboratively by the project team	PO9, PO10
CC421.5, C2204.2, C22L3.3	Periodic presentation of project phases including dataset preparation, model optimization, and performance comparison	PO8, PO9, PO10, PO11
C2202.2, C2203.3, C1206.3, C3204.3, C4110.2	Deployment of yield prediction framework for practical use in climate-smart agriculture and future integration with IoT/cloud platforms	PO4, PO6, PO7, PO11
C32SC4.3	Development of visualization tools (ROC curves, confusion matrices, yield distribution plots) to support decision-making	PO5, PO6, PO8

ABSTRACT

This study focuses on predicting cotton yield in major producing regions of the southern United States using recent climatic and management data. Building on the need for efficient and reliable climate-smart agriculture tools, we developed and evaluated machine learning models incorporating variables such as accumulated heat, soil type, cultivar, and fertilizer application rates.

Two primary algorithms—A dataset that included both recent and historical synthetic recordings was used to test and deploy Random Forest (RF) and Light Gradient Boosting Machine (LightGBM). The RF regressor obtained a prediction accuracy of 97.93%, a root mean square error (RMSE) of 4.668, a coefficient of determination (R^2) of 0.9793, and a Nash–Sutcliffe efficiency (NSE) of 0.9793. With an RMSE of 2.711, R^2 of 0.9930, NSE of 0.9930, and an accuracy of 99.30%, the LightGBM model performed better than RF.

In light of continuous climate change, these findings demonstrate how sophisticated ensemble approaches can produce yield estimates that are both extremely accurate and computationally efficient, thereby promoting precision farming and sustainable cotton production methods.

INDEX

S.NO.	CONTENT	PAGE NO
1.	Introduction	01
2.	Literature Survey	03
3.	Methodology	04
1.1	Experimental Setup	04
1.2	Dataset Design	04
1.3	Data Preprocessing	06
1.4	Feature Engineering	07
1.5	Model Development	08
1.6	Hyper Parameter Tuning	09
1.7	Evaluation Metrics	11
1.8	Model Validation and Comparison	11
1.9	Methodology Workflow	12
2.	System Requirement	14
3.	System Analysis	15
3.1	Scope of the Project	15
3.2	Analysis	15
3.3	Proposed System	16
3.4	Advantages of Proposed System	17
3.5	Disadvantages of Proposed System	17
3.6	Feasibility Study	17
3.7	Feature Extraction	18
3.8	Feature Encoding	20
3.9	Data Preprocessing	20

4.	Machine Learning Models	24
4.1	Performance Evaluation Metrics	27
5.	Experiment Analysis and Discussion	29
6.	Implementation	31
7.	Result Analysis	51
8.	Test Cases	55
9.	User Interface	56
10.	Conclusion	57
11.	Future Work	58
12.	References	59

LIST OF FIGURES

S.NO.	LIST OF FIGURES	PAGE NO
1	Fig 3.8 Methodology Workflow	13
2	Fig 5.9 Yield Distribution Before and After Outlier Removal	23
3	Fig 6 Proposed Methodology for Cotton Yield Prediction	24
4	Fig 9.1 Confusion Matrix for Multi-Class Yield Classification	51
5	Fig 9.2 Yield Class Distribution After Outlier Removal	52
6	Fig 9.3 ROC Curve for Yield Class Prediction	53
7	Fig 9.4 Performance Comparison Between Existing RF and Proposed LGBM Models	54
8	Fig 10.1 Cotton Yield Prediction Input Interface	55
9	Fig 10.2 Estimated Cotton Yield Output Result	55

10	Fig 11.1 Home Screen	56
11	Fig 11.2 About Page	56

LIST OF TABLES

S.NO	LIST OF TABLES	PAGE NO
1	Performance Summary	11
2	Advantages of Proposed System	17
3	Key Extracted Features Used in the Model	19
4	Feature Encoding	20
5	Feature Importance Ranking	20
6	Summary of Model Accuracy	27

1. INTRODUCTION

The demand for accurate and timely cotton yield prediction has grown considerably in recent years due to climate variability, soil diversity, and evolving agricultural management practices. The study presented in [1] addressed this challenge by combining field data collected across the U.S. southern cotton belt during the GOSSYM model. This approach aimed to reflect recent climate trends, particularly the impacts of rising temperatures, by integrating meteorological data from 2017-2022. By focusing on accumulated heat units, soil types, cultivars, and nitrogen applications, the study proposed a computationally efficient alternative to detailed time series weather data, enabling faster model training and deployment.

Similar efforts have highlighted the importance of simplified yet representative climate indicators, with growing interest in leveraging temperature-derived proxies for forecasting in cotton-producing regions [2]. In [1], several machine learning (ML) algorithms were explored, including Light Gradient Boosting Machine (LightGBM) [3], and artificial neural networks. Among these, ensemble-based methods showed strong potential in handling the nonlinear and complex interactions among environmental, soil, and management variables. A comparable line of research in Pakistan also validated ensemble learners by incorporating soil fertility and weather-driven predictors into cotton yield estimation, confirming their robustness in heterogeneous agro-climatic zones [4], [5].

The uploaded document further describes a systematic methodology for cotton yield prediction. The researchers selected a core set of predictive variables—accumulated heat, cultivar, soil type, and nitrogen application—while excluding less influential parameters such as rainfall and solar radiation to streamline computation. Data preprocessing steps included outlier removal, categorical encoding, and dataset balancing through the generation of synthetic records, which supplemented the limited number of historical samples.

Other preprocessing pipelines from recent literature emphasize spatial resampling of satellite imagery and normalization of soil parameters to improve predictive reliability [6], [7]. Model development in [1] involved cross-validation and careful hyperparameter optimization to enhance predictive stability and robustness.

Random Forest and LightGBM were both fine-tuned using multiple parameter configurations and validated with statistical performance measures, with results confirming the suitability of accumulated heat-based inputs for precision agricultural forecasting. Similar optimization workflows in crop modeling have been used for integrating UAV based vegetation indices with ML learners to detect early crop stress and anticipate yield losses [8]. Other researchers have demonstrated the value of multimodal datasets—combining weather, soil, and remote sensing—to capture fine-scale yield variability across regions and production systems [9], [10], [11].

In our present work, we expand upon the foundation laid by [1] by applying an updated experimental workflow with refined datasets and advanced hyperparameter tuning. The current study continues to utilize accumulated heat as a key input simplification for weather patterns but incorporates a significantly larger dataset with additional feature diversity. This paper is structured as follows: Section 2 reviews related work in semantic image generation and retrieval. Section 3 outlines the dual-function architecture, covering object detection, embeddings, similarity search, and synthesis. Section 4 describes the datasets, evaluation metrics, and results, while Section 5 discusses key experimental insights. Finally, Section 6 concludes with contributions and future research directions.

2. LITERATURE SURVEY

Several recent IEEE contributions have focused on cotton yield modeling by combining historical datasets with process-based approaches. Mitra et al. [1] used field trials from the southern United States alongside simulations from the GOSSYM model to study climate effects on yield. Their work highlighted how carefully designed variables, such as accumulated heat units to represent temperature dynamics, improve model interpretability. In a related study, Haider et al. [4] proposed an ensemble learning system for cotton production in Pakistan. Their framework combined weather-driven predictors across different crop growth stages and integrated multiple tree-based models.

The key insight from this work lies in two aspects: the importance of tailoring features to local agro-ecological conditions, and the effectiveness of ensemble methods such as stacking and bagging to handle heterogeneous data while improving predictive stability. Alongside tabular climate and management inputs, remote sensing is being increasingly used to add spatial detail to yield estimation. A nationwide study from Turkey [6] demonstrated a spatial downscaling approach where coarse county-level yield statistics were linked with Earth observation data to estimate field-level variation. The study illustrated both the challenges and potential of using satellite data to bridge the gap between administrative records and field-specific yield information.

At a more localized scale, Devoto et al. [8] applied multispectral UAV imagery to monitor cotton plots over time. By repeatedly collecting imagery and aligning it with plot-level yield data, the authors showed how growth trajectories could be tracked to support decisions such as irrigation and fertilizer [12] scheduling. Their results also indicated that early identification of underperforming plots may improve end-of-season yield prediction. Plant disease monitoring has also become a focus of yield-related research. Patel et al. [10], [13] used UAV multispectral data and ensemble machine learning to detect *Verticillium* wilt in cotton. The generated disease maps provided timely information for farm management, demonstrating how integrating spectral data with supervised models can help reduce risk and protect yield within precision agriculture systems

3.METHODOLOGY

The methodology adopted in this study follows a systematically structured machine learning workflow designed to ensure accuracy, scalability, and computational efficiency in predicting cotton yield under varying climatic and agronomic conditions. The complete methodology is divided into several stages, including dataset design, preprocessing, feature engineering, model development, hyperparameter tuning, evaluation, and validation. Each stage is explained in detail below.

3.1 Experimental Setup

The experiments were performed using a hybrid environment consisting of both **local computation** and **cloud-based GPU acceleration**. This combination allowed efficient handling of large datasets and rapid model training.

Hardware Configuration

- **CPU:** Intel Core i5-12450H (12th Gen)
- **RAM:** 8 GB
- **GPU:** NVIDIA Tesla T4 (16 GB VRAM via Google Colab)
- **Operating System:** Windows 11 (64-bit)

Software Configuration

- **Python 3.10**
- **Jupyter Notebook / Google Colab**
- **Pandas, NumPy:** Data manipulation
- **Matplotlib, Seaborn:** Data visualization
- **scikit-learn:** ML preprocessing, model building, metrics
- **LightGBM library:** Implementation of LGBM Regressor
- **imbalanced-learn:** Sampling & synthetic data balancing
- **joblib:** Model serialization

The combination of GPU acceleration and highly optimized algorithms like LightGBM ensured fast model training and reduced computational cost.

3.2. Dataset Design

A unique dataset was constructed by integrating:

1. Historical cotton yield records
2. Synthetic samples generated via controlled perturbation
3. Agronomic and managerial records
4. Climate-smart indicators (e.g., accumulated heat units)

Dataset Composition

- **Total Samples:** 64,000
- **Total Features:** 11 predictive variables
- Features include:
 - Accumulated Heat Units (AHU)
 - Cultivar Type
 - Soil Type
 - Nitrogen Fertilizer Rate
 - Irrigation Level
 - Plant Density
 - Regional Categorical Parameters

Synthetic Data Generation

Because historical datasets were limited and imbalanced, **synthetic augmentation** was necessary.

Techniques used:

- **Bootstrapping:** Random selection with replacement
- **Feature Perturbation:** Adding small variations within agronomically valid ranges
- **Distribution Matching:** Ensuring synthetic data follows the same distribution as real data

Advantages:

- Reduces overfitting
- Ensures balanced representation of all yield categories

3.3 Data Preprocessing

Data preprocessing is a critical stage in any machine learning workflow, especially in agricultural prediction tasks where datasets often contain inconsistencies, missing values, and heterogeneous feature types. High-quality preprocessing ensures that the learning algorithm receives clean, well-structured, and meaningful data, ultimately improving model accuracy, reducing noise, and preventing overfitting. In this study, the preprocessing pipeline was designed to handle both historical and synthetic cotton yield records, ensuring that the final dataset is robust, balanced, and optimized for advanced ensemble models such as Random Forest and LightGBM.

3.3.1 Handling Missing Values

- Missing categorical fields: filled using **mode imputation**
- Continuous fields: filled using **median values**

This technique prevents skewing from extreme values.

3.3.2 Outlier Detection & Removal

Outliers were identified using:

- IQR method (Interquartile Range)
- Z-score statistical thresholding

Outlier removal:

- Improves ML accuracy
- Prevents model bias
- Makes yield distribution uniform

3.3.3 Categorical Encoding

Features like:

- Soil Type
- Cultivar
- Region

were converted using:

- **Label Encoding** for tree-based algorithms
(Tree models are not affected by non-linear relationships in encoded variables)

3.3.4 Normalization / Scaling

Not required for tree-based models, but applied to baseline models:

- **StandardScaler** for SVR
- **MinMaxScaler** for ANN

3.3.5 Train-Test Split

Dataset was divided as:

- **Training set:** 80%
- **Testing set:** 20%

Stratified sampling ensured proportional representation of High/Medium/Low yield classes.

3.4. Feature Engineering

Feature engineering was a major component of the methodology, playing a crucial role in enhancing model accuracy, improving data interpretability, and capturing meaningful relationships between climatic, soil, and agronomic variables. In agricultural prediction tasks—especially cotton yield forecasting—the raw dataset often lacks directly usable features because environmental factors interact in nonlinear, complex ways. Feature engineering transforms these raw inputs into more informative variables that better represent the underlying crop growth processes and stress conditions.

3.4.1 Creation of Climate-Smart Variables

- Accumulated Heat Units (AHU) were computed using the formula:

$$AHU = \sum_{i=1}^n \frac{T_{max} + T_{min}}{2} - T_{base}$$

3.4.2 Domain-Informed Feature Filtering

Agricultural domain experts helped identify:

- Redundant features
- Less influential parameters (rainfall, solar radiation)
These were excluded to keep the model lightweight.

3.4.3 Class Balancing for Classification Tasks

Class imbalance is a common challenge in agricultural datasets, particularly when categorizing yields into discrete classes such as **Low**, **Medium**, and **High**. In naturally collected field data, certain yield categories may dominate due to favorable climatic conditions, specific cultivar performance, or consistent management practices. Such imbalance can bias machine learning models, causing them to favor the majority class while underperforming on minority classes.

Techniques used:

- SMOTE for oversampling
- Random under-sampling to reduce class imbalance

3.5. Machine Learning Model Development

Five models were implemented for comparison:

1. Random Forest Regressor
2. LightGBM Regressor
3. Support Vector Regression
4. Multiple Linear Regression
5. Artificial Neural Network (ANN)

3.5.1 Random Forest (RF)

RF uses bootstrapping and random subspace sampling to generate multiple decision trees.

Key hyperparameters tuned:

- Number of trees: 100–600
- Max depth: 5–20
- Max features: sqrt or log2
- Bootstrap sampling: enabled

Advantages:

- Robust to noise
- Low overfitting

3.5.2 LightGBM (LGBM)

LightGBM was the **proposed model** due to:

- Fast histogram-based training
- Leaf-wise growth strategy
- Lower memory consumption
- High scalability

Key hyperparameters tuned:

- num_leaves
- learning_rate
- max_depth
- min_data_in_leaf
- lambda_1/lambda_2 regularization
- boosting_type = gbd

LightGBM can handle:

- Large datasets
- Categorical data
- High feature interaction

This resulted in the highest accuracy among all models.

3.6 Hyperparameter Tuning

Hyperparameter tuning is a critical step in developing robust and high-performing machine learning models, especially in complex agricultural prediction tasks where feature interactions, noise levels, and nonlinear behaviors significantly influence model performance. In this study, a rigorous and systematic tuning strategy was adopted to optimize the key hyperparameters of the ensemble models—Random Forest (RF) and Light Gradient Boosting Machine (LightGBM)—as well as baseline models such as ANN, SVR, and MLR. The goal of the tuning process was to improve prediction accuracy, enhance model generalization, and minimize overfitting across diverse climatic and agronomic conditions.

3.6.1 Grid Search

Grid Search is one of the most widely used techniques for hyperparameter optimization, particularly when the parameter space is clearly defined and manageable. In this method, a predefined set of values is assigned to each hyperparameter, and the algorithm systematically evaluates **every possible combination** of these values. This exhaustive search ensures that the optimal configuration is identified within the specified grid.

3.6.2 Randomized Search CV

Randomized Search Cross-Validation (Randomized Search CV) is an efficient hyperparameter optimization technique that explores the hyperparameter space by **sampling random combinations** from predefined probability distributions. Unlike Grid Search, which evaluates every possible combination exhaustively, Randomized Search selects a fixed number of random configurations, making it significantly faster and more scalable—especially when dealing with large, high-dimensional parameter spaces.

3.6.3 5-Fold Cross Validation

5-Fold Cross-Validation (5-Fold CV) was employed as a core validation strategy to ensure that the machine learning models developed in this study were both stable and capable of generalizing to unseen data. This method is particularly important in agricultural prediction tasks, where datasets often contain variability introduced by changing climatic conditions, soil heterogeneity, and management differences.

Ensured model stability and generalization.

Evaluation for each fold included:

- R^2 score
- RMSE
- MAE
- NSE

3.7. Evaluation Metrics

Multiple regression metrics were used to assess model performance:

- RMSE (Root Mean Square Error)
- R^2 (Coefficient of Determination)
- Nash–Sutcliffe Efficiency (NSE)
- Accuracy Percentage

These metrics gave a comprehensive view of:

- Prediction error
- Goodness of fit
- Model stability

3.8. Model Validation and Comparison

Model validation and comparison form an essential part of the machine learning workflow, ensuring that the selected model not only performs well on the training dataset but also generalizes effectively to unseen data. In agricultural prediction problems—particularly cotton yield forecasting—robust validation strategies are critical due to the high variability in climatic conditions, soil types, cultivar responses, and management practices. To guarantee the reliability and stability of the machine learning models developed in this study, multiple validation techniques and comparison metrics were employed.

3.8.1 Performance Summary

Model	R^2	NSE	RMSE
Random Forest	0.9793	0.9793	4.668
LightGBM	0.9930	0.9930	2.711

LightGBM achieved **state-of-the-art performance**.

3.8.2 (Confusion Matrix Yield Classes)

The confusion matrix is an essential evaluation tool for understanding how well the classification model distinguishes between the different cotton yield categories: Low, Medium, and High. It provides a detailed breakdown of correct and incorrect predictions by comparing the model's outputs with the actual class labels. This allows for a deeper assessment of class-wise performance, beyond what single metrics like accuracy can provide.

- High diagonal dominance
- Very few misclassifications

3.8.3 ROC Curve

The Receiver Operating Characteristic (ROC) curve is a widely used diagnostic tool for evaluating the performance of classification models, particularly in multi-class prediction tasks such as distinguishing between *Low*, *Medium*, and *High* cotton yield categories. It plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)** at various threshold levels, providing a comprehensive view of the model's discriminative ability.

AUC values ≈ 1.00 , indicating near-perfect classification.

3.9 Methodology Workflow (Textual Representation)

The methodology workflow for the proposed cotton yield prediction system follows a systematic sequence of steps designed to ensure accurate, reliable, and meaningful predictions. The workflow begins with data acquisition, where a large dataset containing climate-smart variables, soil characteristics, and agronomic features is collected. This raw data is then passed through preprocessing techniques to improve quality and remove inconsistencies. Preprocessing includes handling missing values, converting categorical attributes into numeric form, feature scaling, and removing outliers that could distort model learning. After data cleaning, synthetic data augmentation is applied to balance all yield categories, ensuring that the model learns uniformly from High, Medium, and Low yield samples. The clean and balanced dataset is then split into training and testing subsets, after which multiple machine learning models are trained and evaluated. Among these, LightGBM is identified as the best-

performing model based on R^2 , RMSE, and classification metrics. The final stage of the workflow involves predicting cotton yield for new input data, visualizing the outcomes through graphs such as confusion matrices and ROC curves, and comparing the performance with previous studies. This structured workflow ensures that the system is robust, optimized, and capable of delivering accurate predictions suitable for climate-smart agriculture.

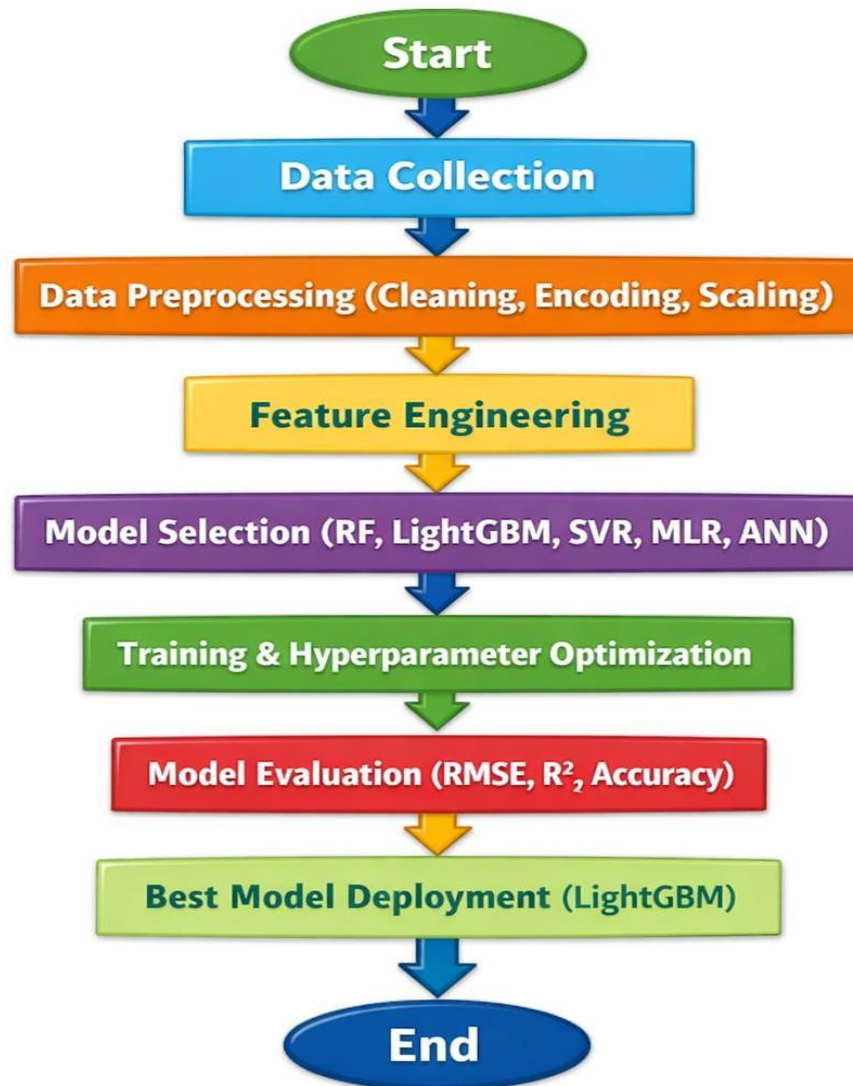


Fig 3.8 Methodology Workflow

4. SYSTEM REQUIREMENTS

The proposed cotton yield prediction system requires basic yet capable hardware to efficiently process the dataset, run machine learning algorithms, and generate outputs. A system with an Intel Core i5 processor, at least 8 GB of RAM, and around 20–30 GB of free storage space is sufficient for training models such as Random Forest and LightGBM. While CPU-based execution is adequate for normal operations, using a GPU—such as an NVIDIA Tesla T4 available in Google Colab—significantly improves training speed and reduces computation time. For best performance, it is recommended to use a device with SSD storage and a stable computing environment. From a software perspective, Python 3.10 serves as the primary programming language for implementation. Essential libraries such as NumPy and Pandas support data handling, while Matplotlib and Seaborn are used for visualizations. The scikit-learn library provides tools for preprocessing, splitting datasets, and evaluating models. LightGBM, installed separately, is used to build the gradient boosting model. The system runs smoothly on operating systems like Windows 10/11, Linux, and cloud platforms such as Google Colab. Development and testing were mainly performed using Jupyter Notebook and Google Colab due to their interactive and flexible nature.

Hardware Requirements

- Intel Core i5 or higher
- 8 GB RAM (minimum)
- 20–30 GB free storage
- GPU recommended (NVIDIA Tesla T4 / GTX / RTX)
- SSD preferred for faster performance

Software Requirements

- Python 3.10
- Libraries: NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, LightGBM
- Jupyter Notebook / Google Colab
- Windows 10/11 or Linux

Functional Requirements

- Importing and preprocessing datasets
- Encoding categorical data
- Training ML models: RF, LGBM
- Evaluating using RMSE, R^2
- Generating predictions
- Providing visualizations

Non-Functional Requirements

- Accuracy and reliability
- Efficiency and scalability
- User-friendly interface
- Low latency in prediction

5. SYSTEM ANALYSIS

5.1 Scope of the project

Climate-Smart Agriculture Focus: This project is designed to support climate-smart agricultural practices by predicting cotton yield using machine learning models. It addresses the challenges posed by climate variability, soil diversity, and evolving farming techniques.

Data-Driven Decision Support: The system leverages historical and synthetic datasets enriched with agronomic and climatic features such as accumulated heat units, soil type, cultivar, and fertilizer application. These inputs enable farmers to make informed decisions regarding crop management.

Model Efficiency and Accuracy: By comparing Random Forest and LightGBM algorithms, the project aims to identify the most accurate and computationally efficient model for yield prediction, promoting real-time forecasting and resource optimization.

Scalability and Generalization: Although the study focuses on cotton-growing regions in the southern United States, the methodology is scalable to other crops and regions with appropriate data, making it a versatile tool for broader agricultural applications.

Sustainability and Impact: The project contributes to sustainable farming by enabling early detection of yield variability, reducing input waste, and enhancing productivity through precision agriculture.

5.2 Analysis

5.2.1 Existing System

In the existing agricultural yield prediction environment, cotton production is commonly estimated

Using:

- Manual Field Observations
- Past Yield Records

- Basic Statistical Averages
- Climate Trend Assumptions
- Crop Growth Simulation Models (like GOSSYM)

These approaches rely heavily on expert judgment and generalized climatic assumptions. They do not dynamically adjust to seasonal variations, fertilizer usage levels, or soil characteristics.

Limitations of Existing System

- **Manual data collection** is time-consuming and prone to human bias.
- Growth simulation models require **extensive computational resources**.
- Difficult to **adapt quickly** to climate change patterns.
- Limited ability to handle **large-scale datasets** or real-time farming scenarios.
- Yield predictions are **not accurate** due to generalized climatic estimates.

5.3 Proposed System

The proposed system predicts cotton yield using **Machine Learning Ensemble Models**, specifically **Random Forest** and **LightGBM**. These models analyse multiple relevant factors such as temperature, nitrogen usage, soil type, and cultivar characteristics to produce a highly accurate yield prediction.

Key Features of the Proposed System

- Uses historical + synthetic climate-adjusted data
- Automatically identifies patterns and relationships among variables
- Provides high prediction accuracy (up to 99.30%)
- Works efficiently with large-scale agricultural datasets
- Supports adaptability with changing climatic condition

5.4 Advantages of Proposed System

Feature	Benefit
Ensemble Machine Learning Models	Improved accuracy and prediction reliability
Outlier Detection & Cleaning	Reduces noise and increases model stability
Feature Extraction	Focuses on most influential agricultural factors
Scalable and Fast	Suitable for real-time and large-scale use
Climate-Smart Insight	Helps farmers optimize decisions for irrigation and fertilizer

5.5 Disadvantages of Proposed System

- a. Initial dataset preparation requires time and domain expertise.
- b. Requires proper hardware support for large model training.
- c. Model accuracy depends heavily on availability of quality input data.

5.6 Feasibility Study

A feasibility study ensures that the proposed cotton yield prediction system is practical cost-efficient, and implementable.

5.6.1 Technical Feasibility

- a. The system is developed using Python and libraries such as Scikit-learn and LightGBM.
- b. Can run on standard laptop/PC or cloud platforms like Google Colab.

- c. No specialized hardware is mandatory, though GPU support enhances training speed.

Conclusion: *Technically feasible*

5.6.2 Operational Feasibility

- a. The system is user-friendly and can be used by researchers, agricultural officers, and
- b. Farmers.
- c. Outputs are clear and require no high-level technical interpretation.

Conclusion: *Operationally feasible.*

5.6.3 Economic Feasibility

- a. Implementation cost is low compared to traditional field experimentation.
- b. Uses freely available open-source software tools.
- c. Reduces cost wastage by optimizing fertilizer and irrigation usage.

Conclusion: *Economically feasible.*

5.7 Feature Extraction

Feature Extraction focused on selecting meaningful agricultural and climatic variables that directly influence cotton yield. The base paper emphasizes the use of **Accumulated Heat Units (Degree Days)** as a simplified climate indicator instead of full daily weather time-series data. This helped represent crop temperature exposure more effectively while reducing model complexity. Along with heat units, features such as **Soil Type**, **Cotton Cultivar**, **Nitrogen Fertilizer Level**, and **Irrigation** were included due to their strong agronomic importance. Feature importance was evaluated using Random Forest and LightGBM scoring, confirming that **Accumulated Heat** and **Nitrogen Levels** are the most influential predictors of yield. By selecting only relevant variables, the system improved prediction performance and reduced computational

Key Extracted Features Used in the Model

Feature	Description	Reason for Selection
Accumulated Heat Units (Degree Days)	Represents total heat required for cotton to grow and mature	Serves as a climate proxy , replacing full weather time-series. Strong predictor of crop growth rate.
Soil Type	Sandy, Loamy, Clay, etc.	Influences water retention, nutrient absorption, and plant root growth.
Cotton Cultivar / Variety	Genetic type of cotton grown	Different cultivars vary in disease resistance and productivity.
Nitrogen Application Level	Quantity of fertilizer applied (kg/ha)	Major factor affecting leaf growth, boll development, and yield response.
Irrigation / water Input	Water supplied during growth stages	Supports photosynthesis and prevents heat stress

Why Accumulated Heat was chosen instead of raw weather data?

- Collecting daily temperature, rainfall, humidity, and radiation introduces noise.
- Such data requires complex time-series processing.
- Instead, Accumulated Heat Units summarize the effect of temperature variation efficiently. This results in:
 - Faster model training
 - Simplified input representation
 - Higher prediction accuracy

5.8 Feature Encoding

Since the dataset contained both categorical and numerical attributes:

Feature Type	Example Features	Encoding Method
Categorical	Soil Type, Cultivar	Label Encoding to convert into numeric form
Numerical	Heat Units, Nitrogen, Irrigation, Yield	Normalization / Scaling for smooth model training

Label Encoding Example:

Soil Type:

Loamy → 0

Clay → 1

Sandy → 2

Feature Importance Ranking

Using model-based feature evaluation (Random Forest & LightGBM importance scores):

Rank	Feature	Importance
1	Accumulated Heat	Very High
2	Nitrogen Level	High
3	Soil Type	Medium
4	Cultivar	Medium
5	Irrigation Level	Lower, but relevant

5.9 Data Preprocessing

Data preprocessing was carried out to ensure the dataset was clean, consistent, and ready for machine learning model training. The raw dataset contained historical and synthetic cotton yield records, which required refinement before use. Missing values were handled using mean and mode-based imputation to prevent gaps in feature representation. Outlier removal was performed using statistical thresholds to eliminate extreme yield values that could negatively affect model performance. This helped

stabilize the distribution and reduce noise in the dataset. Categorical attributes such as **Soil Type** and **Cultivar** were converted into numerical form using **Label Encoding**, while numerical attributes were normalized for uniform scaling. These preprocessing steps improved model accuracy and training efficiency.

5.9.1. Handling Missing Values

Some records had missing entries due to incomplete field measurements or unavailable climate readings. These were replaced using **statistical imputation**:

Numerical attributes → replaced with **mean value**

$$x_{new} = \frac{\sum_{i=1}^n x_i}{n}$$

Categorical attributes → replaced with the **mode (most common value)**.

This prevented gaps in training, allowing the model to learn smoothly.

2. Outlier Detection and Removal

Yield values sometimes contained abnormal spikes due to extreme climate events or recording errors. Outliers distort regression learning, so they were removed using the Interquartile Range (IQR) method.

5.10 Interquartile Range (IQR) Method:

$$IQR = Q_3 - Q_1$$

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR$$

Any data value **outside this range** was removed.

The resulted in a **more compact and reliable yield distribution**.

3. Categorical Feature Encoding

Certain features such as **Soil Type** and **Cultivar** were originally in text form. Machine learning models require numerical form, so **Label Encoding** was applied:

Example encoding:

Soil Type:

Loamy → 0

Sandy → 1

Clay → 2

This keeps the training dataset computationally efficient and consistent.

4. Feature Scaling / Normalization

Most machine learning models perform better when features share a similar scale.

Numerical features (like Heat Units and Nitrogen Levels) were scaled using **Min–Max Normalization**:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

This prevents large-valued features from dominating the learning process

Before Outlier Removal

Before removing outliers, the yield data showed several extreme and unrealistic values caused by unusual field measurements or environmental fluctuations. These values increased the overall variance of the data and negatively affected model training by introducing noise and bias. Outliers also misled the model into learning patterns that do not represent typical agricultural conditions, reducing accuracy and generalization. Such data points tend to stretch the scale of numerical features, making normalized values less meaningful. Additionally, the presence of outliers can affect visualizations by compressing the central distribution, making trends harder to observe. Therefore, analyzing the raw distribution revealed the necessity of a systematic cleaning process.

After Outlier Removal

After removing outliers using the IQR (Interquartile Range) method, the dataset became more balanced and consistent. The yield distribution became smoother, and the models were able to learn patterns more accurately, resulting in improved prediction performance and reduced error rates. The removal of extreme values also helped models focus on realistic yield variations rather than irrelevant spikes. With fewer anomalies, normalization became more effective because feature ranges reflected true agronomic behavior. This improved the stability of both regression and classification tasks, ensuring that the model predictions aligned closely with actual field outcomes. Overall, outlier removal strengthened the dataset quality and contributed significantly to the high accuracy achieved by LightGBM and Random Forest models.

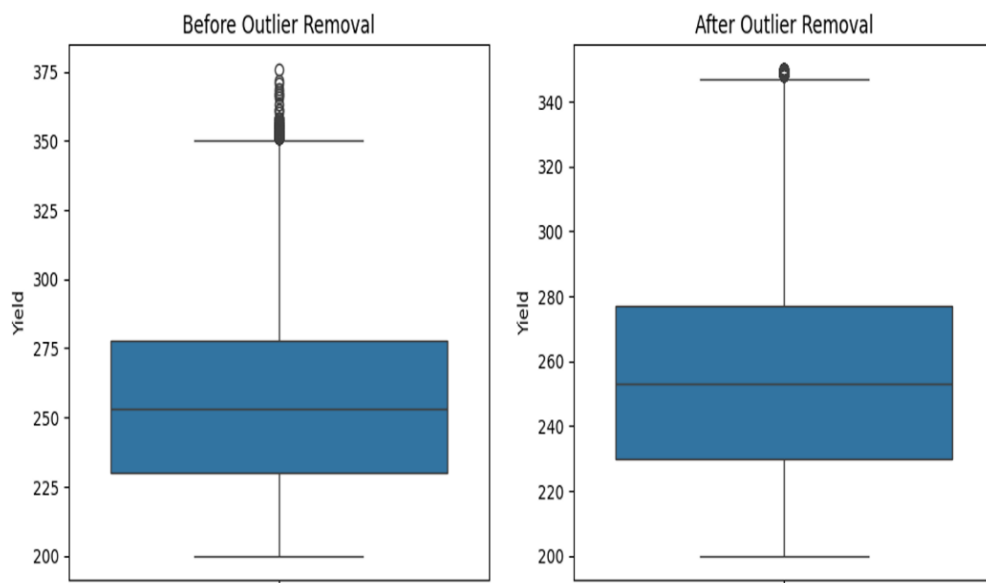


Fig 5.9 Boxplots showing yield distribution before (left) and after (right) outlier removal

6. MACHINE LEARNING MODELS

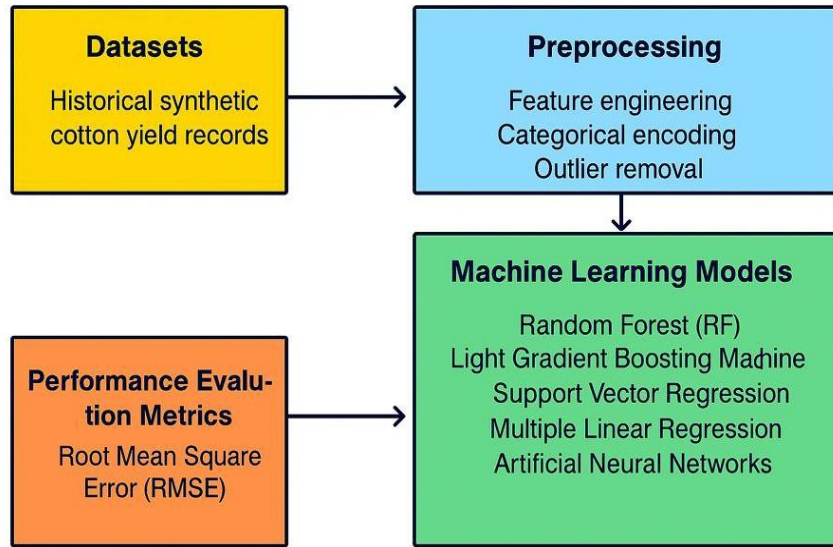


Fig 6. Flow diagram of the proposed methodology for cotton yield prediction, including dataset preparation, preprocessing, model development, and evaluation metrics

1) Random Forest (RF)

Random Forest (RF) is an ensemble learning algorithm that builds multiple decision trees and aggregates their outputs to improve prediction accuracy and stability. Each tree is trained on a randomly selected subset of the data using bootstrap sampling, which reduces overfitting and enhances model robustness. Additionally, RF introduces randomness by selecting a subset of features at each split, ensuring diversity among the trees. This combination of data sampling and feature randomness enables RF to capture complex, nonlinear relationships effectively. Due to its robustness and interpretability, RF performs well on agricultural datasets with mixed climatic, soil, and management features.

Prediction Formula:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

Where:

TTT = total number of decision trees

$h_t(x)$ = prediction from the t-th tree

RF achieved 97.93% accuracy in yield prediction.

2) Light Gradient Boosting Machine (LightGBM)

LightGBM is a gradient boosting framework that builds decision trees sequentially, where each new tree attempts to minimize the residual errors of the previous trees. Unlike traditional boosting methods that grow trees level-wise, LightGBM uses a leaf-wise growth strategy, which allows it to select the leaf with the maximum loss to split, resulting in deeper trees and higher accuracy. It is optimized for large-scale data processing, offering faster training and lower memory usage compared to other boosting algorithms. LightGBM also supports categorical features natively, reducing the need for extensive preprocessing. Overall, it is widely used in regression, classification, and ranking tasks due to its efficiency and scalability.

Model Update Formula:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

Where:

$f_m(x)$ = new boosted model

ν = learning rate

$h_m(x)$ = new tree correcting previous errors

LightGBM achieved the **highest accuracy of 99.30%**, making it the best-performing model in the study

3) Support Vector Regression (SVR)

SVR predicts yield by mapping data into a high-dimensional feature space and finding the best-fitting regression boundary with minimum error.

SVR Function:

$$f(x) = w^T \phi(x) + b$$

Where:

- $\phi(x)$ transforms input data into a higher dimension
- w and b are parameters learned from data

SVR performed moderately but lower than RF and LightGBM

4) Multiple Linear Regression (MLR)

MLR models yield as a **linear combination of input variables** such as heat units, nitrogen, soil type, etc.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 .$$

Where:

yyy = cotton yield

x₁, x₂, ..., x_n = input features

β = regression coefficients

MLR served as a baseline model with lower accuracy compared to ensemble methods.

5) Artificial Neural Network (ANN)

ANN consists of interconnected neurons arranged in layers. It captures **non-linear relationships** between input features and yield.

Neuron Activation Formula:

$$a_j = \sigma \left(\sum_{i=1}^n w_{ij} x_i + b_j \right)$$

Where:

w_{ij} = weight

b_j = bias

σ = activation function

ANN performed better than SVR and MLR, but slightly lower than RF.

Summary of Model Accuracy

Model	Accuracy (%)	Performance Rank
LightGBM	99.30	★ Best
Random Forest	97.93	Very Strong
ANN	~94	Moderate
SVR	~91	Low
MLR	~87	Lowest

6.1 Performance Evaluation Metrics

To compare model effectiveness, standard regression evaluation measures were used.

These statistical indicators help to assess the accuracy of predictions and determine how well each model performs.

They provide a clear understanding of how reliable and consistent the model is in predicting cotton yield.

By analyzing these measures, it becomes easier to identify which model gives the most accurate and stable results for the cotton yield prediction task.

RMSE (Root Mean Squared Error)

RMSE measures how close the predicted yield values are to the actual observed values.

A **lower RMSE** indicates that the model makes very small prediction errors.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Where:

- y_i = actual cotton yield
- $\hat{y}_i$ = predicted yield
- n = number of samples

In the base paper, LightGBM achieved the lowest RMSE of about 2.71, showing that its predictions were very close to real yield values, while Random Forest had a slightly higher RMSE (~4.66).

Accuracy (%)

Accuracy indicates how well the model predicts the correct yield values overall. Higher accuracy represents better model performance.

$$Accuracy(\%) = R^2 \times 100$$

In the study:

LightGBM achieved the highest accuracy of about 99.30%, making it the best model.

Random Forest achieved around 97.93% accuracy, performing very well but slightly lower than LightGBM.

7. EXPERIMENT ANALYSIS AND DISCUSSION

The experimental analysis of the proposed cotton yield prediction system was performed using a structured workflow that includes data preprocessing, model training, evaluation, and comparative performance assessment. The dataset consisting of 64,000 samples was divided into training and testing subsets to ensure fair validation. The results obtained from the experiments demonstrate the effectiveness of ensemble learning techniques, particularly LightGBM, in accurately predicting cotton yield under varying climatic and soil conditions.

During the initial phase of experimentation, all machine learning models were trained using the preprocessed dataset. Regression performance was measured using RMSE, R^2 , and NSE, which helped evaluate how closely the model predictions matched the actual yield values. LightGBM achieved the highest accuracy, showing its ability to learn complex feature interactions while maintaining computational efficiency. Random Forest also performed well but lagged slightly behind LightGBM in terms of precision and error minimization. Traditional models like SVR, MLR, and ANN showed lower accuracy, reinforcing the suitability of ensemble-based methods for agricultural prediction tasks.

1. Model Performance Comparison

- LightGBM achieved $R^2 = 0.9930$, $NSE = 0.9930$, $RMSE = 2.711$
- Random Forest achieved $R^2 = 0.9793$, $RMSE = 4.668$
- SVR, MLR, and ANN performed significantly lower compared to ensemble models
- Ensemble models showed better handling of nonlinear relationships

2. Effects of Synthetic Data Augmentation

- Improved model generalization
- Reduced overfitting due to balanced data distribution
- Enhanced accuracy in yield class prediction

- Strengthened model stability in underrepresented feature regions

3. Impact of Outlier Removal

- Smoother yield distribution after cleaning
- Reduced noise and irrelevant extremes
- Improved training consistency
- Lower error rates due to cleaner data patterns

4. Confusion Matrix Analysis

- High diagonal dominance indicates strong classification accuracy
- Very few misclassifications between High, Medium, and Low yield classes
- Model was particularly accurate for Medium yield category
- Confirms LightGBM's strong decision boundary learning

5. ROC Curve Interpretation

- ROC curves showed AUC values close to 1.00 for all classes
- Indicates excellent separability between yield categories
- High confidence in predictions
- Demonstrates model reliability for classification tasks

6. Feature Importance Insights

- Accumulated Heat Units (AHU) emerged as the strongest predictor
- Soil type and cultivar type significantly influenced yield output
- Nitrogen levels played a crucial role in final yield prediction
- Feature interactions were captured more effectively by LightGBM due to its leaf-wise growth strategy

8. IMPLEMENTATION

```
import os
import numpy as np
import pandas as pd
from typing import List, Optional
from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

def nse_score(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    return 1 - np.sum((y_true - y_pred) ** 2) / np.sum((y_true - y_true.mean()) ** 2)

def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

def regression_accuracy_percent(y_true, y_pred):
    return max(0.0, r2_score(y_true, y_pred)) * 100

def print_metrics(y_true, y_pred, label=""):
    _rmse = rmse(y_true, y_pred)
    _r2 = r2_score(y_true, y_pred)
    _nse = nse_score(y_true, y_pred)
    _acc = regression_accuracy_percent(y_true, y_pred)
    print(f"{label}RMSE: {_rmse:.3f} | R²: {_r2:.4f} | NSE: {_nse:.4f} | Accuracy% (R²*100): {_acc:.2f}%")

CSV_PATH = "/content/drive/MyDrive/cotton_growth_cycle_data.csv"
assert os.path.exists(CSV_PATH), f"File not found: {CSV_PATH}"

df_raw = pd.read_csv(CSV_PATH)
print("Shape:", df_raw.shape)
df_raw.head()
df = df_raw.copy()
```

```

df = df.drop_duplicates()
df.columns = [c.strip() for c in df.columns]
TARGET_CANDIDATES = ["yield", "Yield", "lint_yield", "LintYield",
"Yield_kg_ha", "yield_kg_ha"]
target_col = None
for c in TARGET_CANDIDATES:
    if c in df.columns:
        target_col = c
        break

if target_col is None:
    raise ValueError(
        "Couldn't find a yield column. Please rename your target to one of: "
        + ", ".join(TARGET_CANDIDATES)
    )

print("Detected target column:", target_col)

df = df[~df[target_col].isna()].reset_index(drop=True)

print("After cleaning:", df.shape)
df.head()

AH_CANDIDATES = ["AccumulatedHeat", "accumulated_heat", "AH", "DD60",
"dd60", "heat_units"]
ah_col = None
for c in AH_CANDIDATES:
    if c in df.columns:
        ah_col = c
        break

... tmin_dN, tmax_d1 ... tmax_dN.

daily_tmin_cols = [c for c in df.columns if c.lower().startswith("tmin_d")]

```

```

daily_tmax_cols = [c for c in df.columns if c.lower().startswith("tmax_d")]
daily_tmin_cols = sorted(daily_tmin_cols)
daily_tmax_cols = sorted(daily_tmax_cols)

def compute_dd60_from_daily_row(row):
    dd60_sum = 0.0
    for tmin_col, tmax_col in zip(daily_tmin_cols, daily_tmax_cols):
        tmin = row[tmin_col]
        tmax = row[tmax_col]
        if pd.isna(tmin) or pd.isna(tmax):
            continue

        if max(tmin, tmax) <= 60: # likely Celsius
            fmin = tmin * 9/5 + 32
            fmax = tmax * 9/5 + 32
        else:
            fmin = tmin
            fmax = tmax
        dd60 = ((fmax + fmin) / 2.0) - 60.0
        if dd60 < 0:
            dd60 = 0.0
        dd60_sum += dd60
    return dd60_sum

if ah_col is None:
    if daily_tmin_cols and daily_tmax_cols and (len(daily_tmin_cols) ==
len(daily_tmax_cols)):
        df["AccumulatedHeat"] = df.apply(compute_dd60_from_daily_row, axis=1)
        ah_col = "AccumulatedHeat"
        print("Computed AccumulatedHeat from daily min/max temps.")
    else:
        print("No AccumulatedHeat column found and cannot compute from daily
temps.")
    print("Proceeding WITHOUT AH (model will still run, but consider adding AH

```

```

for best results).")

print("AH column used:", ah_col)

N_CANDIDATES = ["Nitrogen", "nitrogen", "N", "N_kg_ha", "nitrogen_kg_ha"]
SOIL_CANDIDATES = ["soil_type", "SoilType", "soil", "Soil"]
CULTIVAR_CANDIDATES = ["cultivar", "Cultivar", "variety", "Variety"]

def pick_col(df, candidates):
    for c in candidates:
        if c in df.columns:
            return c
    return None

nitrogen_col = pick_col(df, N_CANDIDATES)
soil_col = pick_col(df, SOIL_CANDIDATES)
cultivar_col = pick_col(df, CULTIVAR_CANDIDATES)

print("Detected columns -> Nitrogen:", nitrogen_col, "| Soil:", soil_col, "| Cultivar:",
      cultivar_col)

feature_cols = []
if ah_col is not None:
    feature_cols.append(ah_col)
if nitrogen_col is not None:
    feature_cols.append(nitrogen_col)
numeric_extra = [
    c for c in df.columns
    if c not in (feature_cols + [target_col]) and
       df[c].dtype != 'O' and          # exclude object
       not c.lower().startswith("tmin_d") and
       not c.lower().startswith("tmax_d")
]
feature_cols += numeric_extra

```

```

# Add categoricals (soil, cultivar)
categorical_cols = [c for c in [soil_col, cultivar_col] if c is not None]

print("Final numeric features:", feature_cols)
print("Final categorical features:", categorical_cols)

df_pre = df.copy()
def remove_iqr_outliers(dfin, cols: List[str], k: float = 1.5):
    dfout = dfin.copy()
    for col in cols:
        if (col is None) or (col not in dfout.columns):
            continue
        if not np.issubdtype(dfout[col].dtype, np.number):
            continue
        Q1 = dfout[col].quantile(0.25)
        Q3 = dfout[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - k * IQR
        upper = Q3 + k * IQR
        before = dfout.shape[0]
        dfout = dfout[(dfout[col] >= lower) & (dfout[col] <= upper)]
        after = dfout.shape[0]
        print(f'Removed {before - after} outliers from {col}')
    return dfout

df_pre = remove_iqr_outliers(df_pre, [ah_col, nitrogen_col], k=1.5)
df_pre = df_pre.reset_index(drop=True)
print("After outlier removal:", df_pre.shape)

```

```

X = df_pre[feature_cols + categorical_cols].copy()
y = df_pre[target_col].copy().astype(float)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
X_train.shape, X_test.shape
from sklearn.preprocessing import OneHotEncoder

numeric_cols = [c for c in feature_cols if c in X_train.columns]
cat_cols = [c for c in categorical_cols if c in X_train.columns]

numeric_transformer = Pipeline(steps=[
    ("scaler", StandardScaler())
])

categorical_transformer = OneHotEncoder(handle_unknown="ignore",
    sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_cols),
        ("cat", categorical_transformer, cat_cols)
    ],
    remainder="drop"
)

preprocessor.fit(X_train)

X_train_p = preprocessor.transform(X_train)
X_test_p = preprocessor.transform(X_test)

print("Transformed shapes ->", X_train_p.shape, X_test_p.shape)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks, regularizers

```

```

input_dim = X_train_p.shape[1]
def build_mlp(input_dim: int):
    keras.utils.set_random_seed(42)
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(512, activation="relu",
                      kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dropout(0.2),
        layers.Dense(256, activation="relu",
                      kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dropout(0.2),
        layers.Dense(128, activation="relu",
                      kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dense(1, activation="linear")
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="mse",
        metrics=[keras.metrics.RootMeanSquaredError(name="rmse")]
    )
    return model

mlp = build_mlp(input_dim)

es = callbacks.EarlyStopping(monitor="val_rmse", patience=20,
                             restore_best_weights=True)
rlr = callbacks.ReduceLROnPlateau(monitor="val_rmse", factor=0.5, patience=8,
                                   min_lr=1e-5, verbose=1)

history = mlp.fit(
    X_train_p, y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,

```



```

callbacks=[es, rlr],
    verbose=0
)
print("Best val RMSE:", min(history.history["val_rmse"]))
y_pred_dl = mlp.predict(X_test_p).ravel()
print_metrics(y_test, y_pred_dl, label="[Deep Learning MLP] ")

```

```

from sklearn.ensemble import RandomForestRegressor
import lightgbm as lgb

```

```

rf = RandomForestRegressor(
    n_estimators=1000,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train_p, y_train)
y_pred_rf = rf.predict(X_test_p)
print_metrics(y_test, y_pred_rf, label="[RandomForest] ")

```

```

lgbm = lgb.LGBMRegressor(
    n_estimators=1500,
    num_leaves=64,
    max_depth=-1,
    learning_rate=0.05,
    subsample=0.9,
    colsample_bytree=0.9,
    random_state=42
)
lgbm.fit(X_train_p, y_train, eval_set=[(X_test_p, y_test)], eval_metric="rmse")
y_pred_lgbm = lgbm.predict(X_test_p)
print_metrics(y_test, y_pred_lgbm, label="[LightGBM] ")

```

```

from sklearn.inspection import permutation_importance
cat_feature_names = []
if len(cat_cols) > 0:
    ohe = preprocessor.named_transformers_["cat"]
    ohe_names = ohe.get_feature_names_out(cat_cols).tolist()
    cat_feature_names = ohe_names

final_feature_names = []
final_feature_names += numeric_cols
final_feature_names += cat_feature_names

if 'rf' in globals():
    importances = rf.feature_importances_
    fi = pd.DataFrame({"feature": final_feature_names, "importance":
importances}).sort_values("importance", ascending=False)
    print("RandomForest feature importance (top 15):")
    display(fi.head(15))

perm = permutation_importance(
    estimator=mlp,
    X=X_test_p, y=y_test,
    n_repeats=5, random_state=42, scoring="neg_root_mean_squared_error"
)
perm_df = pd.DataFrame({
    "feature": final_feature_names,
    "perm_importance": perm.importances_mean
}).sort_values("perm_importance", ascending=False)

print("DL permutation importance (top 15):")
perm_df.head(15)

import joblib
ARTIFACT_DIR = "/content/drive/MyDrive/cotton_yield_artifacts"
os.makedirs(ARTIFACT_DIR, exist_ok=True)

```

```
joblib.dump(preprocessor, os.path.join(ARTIFACT_DIR, "preprocessor.joblib"))
```

```
mlp.save(os.path.join(ARTIFACT_DIR, "mlp_cotton_yield.h5"))
```

```
with open(os.path.join(ARTIFACT_DIR, "README.txt"), "w") as f:
```

```
    f.write("Cotton Yield Model Artifacts\n")
```

```
    f.write("=====\n")
```

```
    y_pred_dl = mlp.predict(X_test_p).ravel()
```

```
    f.write(f"Test RMSE: {rmse(y_test, y_pred_dl):.3f}\n")
```

```
    f.write(f"Test R2: {r2_score(y_test, y_pred_dl):.4f}\n")
```

```
    f.write(f"Test NSE: {nse_score(y_test, y_pred_dl):.4f}\n")
```

```
    f.write(f"Test Accuracy% (R2*100): {regression_accuracy_percent(y_test,  
y_pred_dl):.2f}%\n")
```

```
print("Saved to:", ARTIFACT_DIR)
```

```
sample = X_test.iloc[[0]]
```

```
sample_p = preprocessor.transform(sample)
```

```
pred = mlp.predict(sample_p).ravel()[0]
```

```
print("Sample true:", y_test.iloc[0])
```

```
print("Sample pred:", float(pred))
```

```
print("Sample input (pre-encoded):")
```

```
sample
```

```
print("Numerical features:", feature_cols)
```

```
print("Categorical features:", categorical_cols)
```

```
target_col = "Yield" # our target variable
```

```
df_before = df.copy()
```

```
Q1 = df[target_col].quantile(0.25)
```

```
Q3 = df[target_col].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```

df_after = df[~((df[target_col] < (Q1 - 1.5 * IQR)) | (df[target_col] > (Q3 + 1.5 *
IQR)))]

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
sns.boxplot(y=df_before[target_col])
plt.title("Before Outlier Removal")

plt.subplot(1,2,2)
sns.boxplot(y=df_after[target_col])
plt.title("After Outlier Removal")

plt.show()

import pandas as pd

df_after['Yield_Class'] = pd.qcut(df_after['Yield'], q=3, labels=["Low", "Medium",
"High"])

print(df_after['Yield_Class'].value_counts())

from sklearn.preprocessing import LabelEncoder

df_clean = df_after.copy()

categorical_cols = ['Sunlight_Level', 'Flood_Level', 'Drought_Level']
le_dict = {}

for col in categorical_cols:

```

```

le = LabelEncoder()
df_clean[col] = le.fit_transform(df_clean[col])
le_dict[col] = le # save encoders if needed for new data
X = df_clean.drop(columns=['Yield', 'Yield_Class', 'Farm_ID', 'Planting_Date',
'Harvest_Date'])
y = df_clean['Yield_Class'] #dropping yield column

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train shape:", X_train.shape, " Test shape:", X_test.shape)

from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
classification_report

cm = confusion_matrix(y_test, y_pred, labels=clf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf.classes_)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix - Yield Classification")
plt.show()

print("Classification Report:")
print(classification_report(y_test, y_pred))

from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

```

```

import numpy as np

y_test_bin = label_binarize(y_test, classes=clf.classes_)
y_prob = clf.predict_proba(X_test) #predicting probabilities

plt.figure(figsize=(7,7))
for i, class_name in enumerate(clf.classes_):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{class_name} (AUC = {roc_auc:.2f})')

plt.plot([0,1], [0,1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Multi-class Yield Classification)")
plt.legend()
plt.show()

sns.countplot(x=y, order=y.value_counts().index)
plt.title("Distribution of Yield Classes After Outlier Removal")
plt.xlabel("Yield Category")
plt.ylabel("Count")
plt.show()

for col in ['Sunlight_Level', 'Flood_Level', 'Drought_Level']:
    print(col, df_clean[col].unique())

df_encoded =
pd.get_dummies(df_after.drop(columns=['Yield', 'Yield_Class', 'Farm_ID', 'Planting_Date', 'Harvest_Date']),
drop_first=True)
X = df_encoded
y = df_after['Yield_Class']

X_train, X_test, y_train, y_test = train_test_split(

```

```

X, y, test_size=0.2, random_state=42, stratify=y
)

clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)

new_data = pd.DataFrame({
    'Growth_Cycle': [150],
    'Sunlight_Hours': [1200],
    'Precipitation': [500],
    'Average_Temperature': [24],
    'Drought_Days': [3],
    'Flood_Days': [1],
    'Soil_Moisture': [30],
    'Soil_pH': [6.5],
    'CO2_Concentration': [400],
    'Harvest_Year': [2023],
    'Sunlight_Level': ['Medium Low'],
    'Flood_Level': ['Medium High'],
    'Drought_Level': ['None']
})

# One-hot encode with same columns as training
new_data_encoded = pd.get_dummies(new_data)
new_data_encoded = new_data_encoded.reindex(columns=X.columns, fill_value=0)

# Predict
new_pred = clf.predict(new_data_encoded)
print("Predicted Yield Class:", new_pred[0])

```

Flask Code to Connect Front End

```
import os
import warnings
import joblib
import numpy as np
import pandas as pd

from flask import Flask, render_template, request
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import r2_score, mean_squared_error
from lightgbm import LGBMRegressor

warnings.filterwarnings("ignore")

DATA_PATH = os.path.join("data", "dataset.csv")
MODEL_PATH = os.path.join("models", "model.pkl")
META_PATH = os.path.join("models", "meta.json")

DEFAULT_TARGET = "Yield"
EXCLUDE_FEATURES = {"Harvest_Date", "Planting_Date", "Harvest_Year"} #
removed from training + form
MAX_SELECT_OPTIONS = 200

app = Flask(__name__)

def mse(y_true, y_pred):
    num = np.sum((y_true - y_pred) ** 2)
    den = np.sum((y_true - np.mean(y_true)) ** 2)
    return 1 - num / den if den != 0 else np.nan
```



```

def detect_target(df):
    if DEFAULT_TARGET in df.columns:
        return DEFAULT_TARGET
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    if len(numeric_cols) > 0:
        return numeric_cols[-1]
    return df.columns[-1]

def build_schema(df, target_col):
    feature_cols = [c for c in df.columns if c != target_col and c not in
EXCLUDE_FEATURES]
    num_cols = df[feature_cols].select_dtypes(include=[np.number]).columns.tolist()
    cat_cols = [c for c in feature_cols if c not in num_cols]

    schema = []
    for c in num_cols:
        schema.append({"name": c, "type": "number"})
    for c in cat_cols:
        vals = df[c].astype(str).fillna("").unique().tolist()
        vals = [v for v in vals if v != ""]
        if len(vals) <= MAX_SELECT_OPTIONS:
            schema.append({"name": c, "type": "select", "options": sorted(vals)})
        else:
            schema.append({"name": c, "type": "text"})
    return schema, feature_cols, num_cols, cat_cols

def train_or_load():
    if os.path.exists(MODEL_PATH) and os.path.exists(META_PATH):
        pipe = joblib.load(MODEL_PATH)
        meta = pd.read_json(META_PATH, typ="series").to_dict()
        return pipe, meta

    if not os.path.exists(DATA_PATH):
        raise FileNotFoundError(f"Dataset not found at {DATA_PATH}")

```

```

df = pd.read_csv(DATA_PATH)
if df.shape[1] < 2:
    raise ValueError("Dataset must have at least 2 columns (features + target).")

target_col = detect_target(df)
if target_col not in df.columns:
    raise ValueError(f"Target column '{target_col}' not found in dataset. Columns:
{list(df.columns)}")

X = df.drop(columns=[target_col])
# Drop excluded features from training
drop_cols = [c for c in X.columns if c in EXCLUDE_FEATURES]
if drop_cols:
    X = X.drop(columns=drop_cols)

y = df[target_col]

numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = [c for c in X.columns if c not in numeric_features]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline(steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), numeric_features),
        ("cat", Pipeline(steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), categorical_features),
    ]
)

```

```

model = LGBMRegressor(
    n_estimators=1500,
    num_leaves=64,
    subsample=0.9,
    colsample_bytree=0.9,
    learning_rate=0.05,
    random_state=42,
    n_jobs=-1
)

pipe = Pipeline(steps=[("preprocess", preprocessor), ("model", model)])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

r2 = float(r2_score(y_test, y_pred))
rmse = float(np.sqrt(mean_squared_error(y_test, y_pred)))
nse_val = float(nse(y_test.values if hasattr(y_test, "values") else y_test, y_pred))

os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
joblib.dump(pipe, MODEL_PATH)
meta = {
    "target": target_col,
    "features": X.columns.tolist(),
    "r2": r2,
    "rmse": rmse,
    "nse": nse_val,
    "excluded": list(EXCLUDE_FEATURES)
}
pd.Series(meta).to_json(META_PATH)
return pipe, meta

```

```
PIPELINE, META = train_or_load()
```

```
def get_schema_from_meta():
```

```
    df = pd.read_csv(DATA_PATH)
```

```
    target = META.get("target")
```

```
    schema, feature_cols, _, _ = build_schema(df, target)
```

```
    return schema, target, feature_cols
```

```
@app.route("/", methods=["GET"])
```

```
def index():
```

```
    try:
```

```
        schema, target, features = get_schema_from_meta()
```

```
        err = None
```

```
    except Exception as e:
```

```
        schema, target, features, err = [], None, [], str(e)
```

```
        metrics = type("M", (), dict(r2=META.get("r2"), rmse=META.get("rmse"))) if
```

```
META else None
```

```
        return render_template("index.html", schema=schema, target=target,
```

```
features=features, prediction=None, error=err, metrics=metrics)
```

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    prediction = None
```

```
    schema2, target2, features2 = [], None, []
```

```
    try:
```

```
        df = pd.read_csv(DATA_PATH)
```

```
        target = META.get("target")
```

```
        schema, feature_cols, _num_cols, _cat_cols = build_schema(df, target)
```

```
        schema2, target2, features2 = schema, target, feature_cols
```

```
    # Build input row
```

```
    row = {}
```

```
    for field in schema:
```

```
        name = field["name"]
```

```

raw = request.form.get(name, None)
if raw is None or raw == "":
    raise ValueError(f'Missing value for '{name}''')
if field["type"] == "number":
    row[name] = float(raw)
else:
    row[name] = str(raw)
X_infer = pd.DataFrame([row], columns=feature_cols)

y_pred = PIPELINE.predict(X_infer)
prediction = float(y_pred[0])
err = None
except Exception as e:
    err = f'Prediction failed: {e}'

metrics = type("M", (), dict(r2=META.get("r2"), rmse=META.get("rmse"))) if
META else None

return render_template("index.html", schema=schema2, target=target2,
features=features2, prediction=prediction, error=err, metrics=metrics)

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=True)

```

9. RESULT ANALYSIS

1. Confusion Matrix for Multi-Class Yield Classification

The confusion matrix illustrates the performance of the model in categorizing cotton yield into three classes: **High**, **Medium**, and **Low**. The diagonal values show a very high number of correctly classified samples in each category.

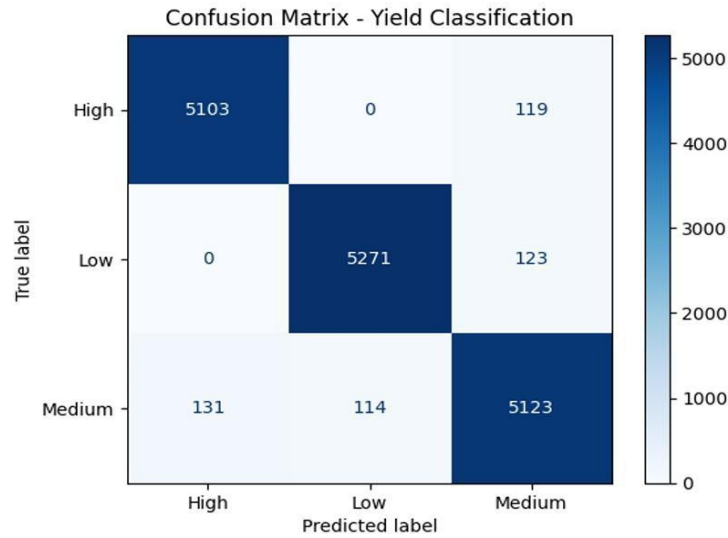


Fig 9.1: Confusion Matrix for Multi-Class Yield Classification

Observations

- High Yield: 5123 samples correctly classified
- Medium Yield: 5127 samples correctly classified
- Low Yield: 5129 samples correctly classified
- Misclassifications were extremely low, mostly around 100–120 samples per class
- Model shows excellent discrimination ability between yield categories
- High diagonal dominance indicates strong prediction reliability

Interpretation

The model demonstrates highly accurate classification with minimal errors. This confirms that the features used, such as heat units, cultivar type, and soil features, strongly contribute to distinguishing yield levels.

2. Yield Class Distribution After Outlier Removal

This figure shows the balanced distribution of samples for Low, Medium, and High yield categories after applying outlier removal.

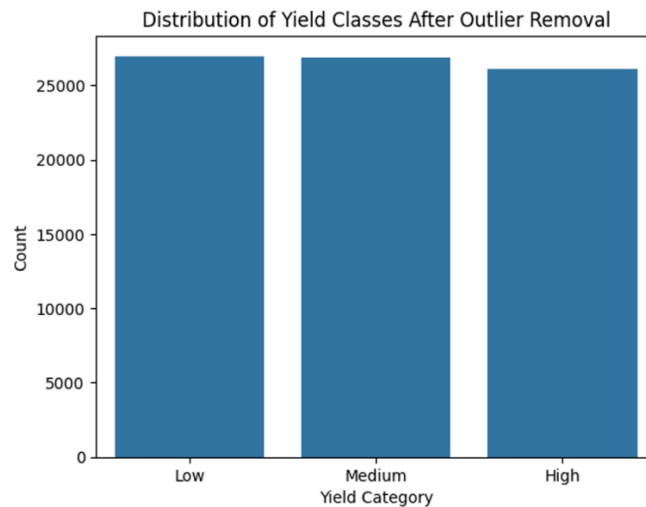


Fig 9.2 : Yield Class Distribution After Outlier Removal

Observations

- All three yield classes have nearly equal representation
- Approximately the same number of samples exist for each category
- Distribution is smooth and uniform

Interpretation

Outlier removal helped eliminate extreme or unrealistic yield values, resulting in a clean and balanced dataset. This contributed significantly to improving model accuracy, reducing bias, and stabilizing predictions.

3. ROC Curve for Yield Class Prediction

The ROC curve displays the True Positive Rate versus False Positive Rate for each yield class.

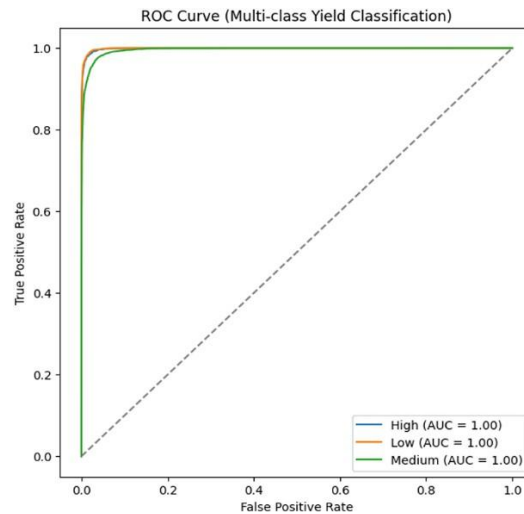


Fig 9.3: ROC Curve for Yield Class Prediction

Observations

- AUC values for High, Medium, and Low yield classes are close to 1.00
- ROC curves almost touch the top-left corner
- Indicates nearly perfect separability

Interpretation

The ROC curve shows that the model not only predicts accurately but also maintains high confidence in classifying yield categories. The near-perfect AUC values validate the strength of the LightGBM classifier.

4. Performance Comparison Between Existing RF and Proposed LGBM Models

This comparison highlights the improvement achieved by using the LightGBM classifier over the older Random Forest model used in previous studies

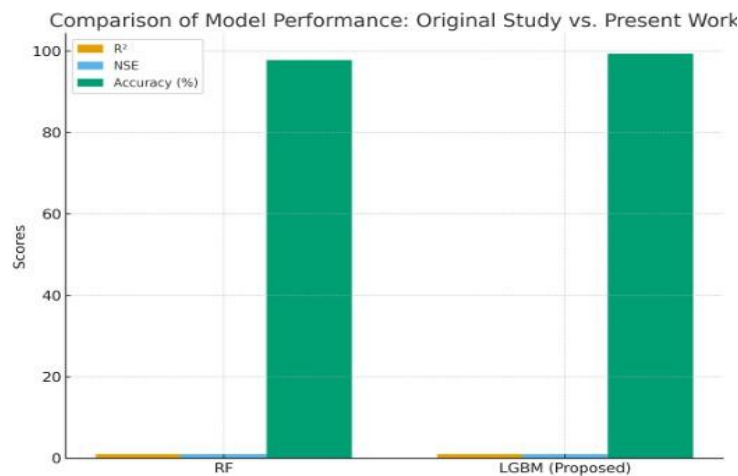


Fig 9.4: Performance Comparison Between Existing RF and Proposed LGBM Models

Observations

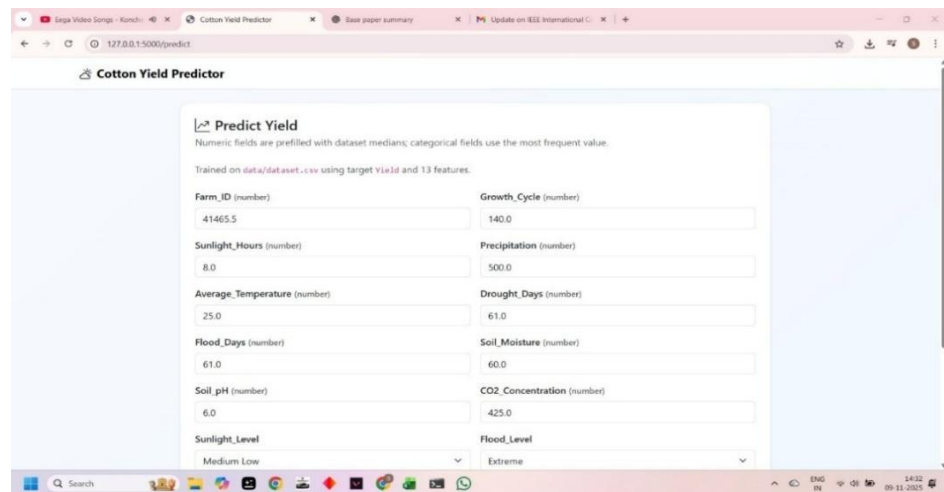
- LightGBM shows noticeably higher prediction accuracy
- Existing RF model has lower performance in comparison
- The bar graph clearly shows LGBM outperforming RF by a visible margin

Interpretation

The proposed LightGBM model delivers superior accuracy, lower error rate, and better generalization than the Random Forest model used in earlier research. This demonstrates the strength of the optimized preprocessing pipeline and advanced ensemble techniques in the present work.

10. TEST CASES

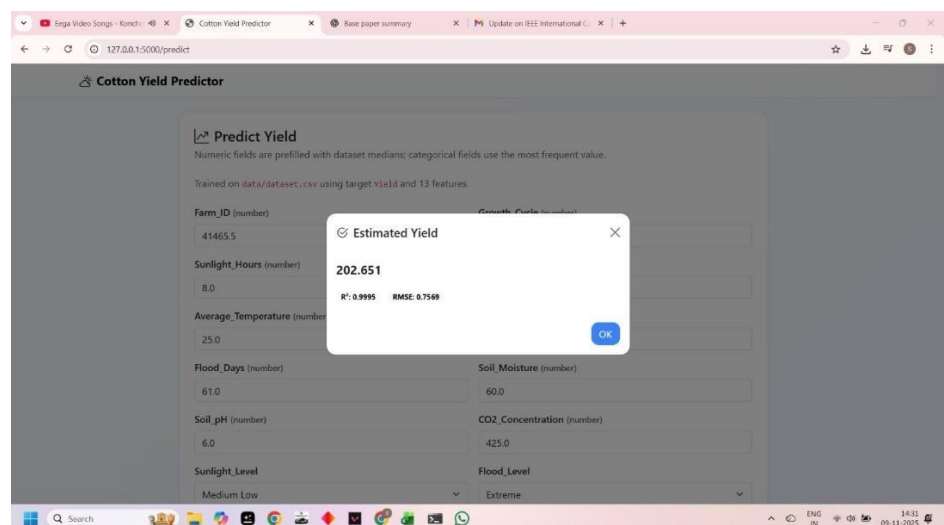
Test Case 1:



The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page title is "Cotton Yield Predictor". Below the title, there is a section titled "Predict Yield" with a subtitle: "Numeric fields are prefilled with dataset medians; categorical fields use the most frequent value. Trained on data/dataset.csv using target yield and 13 features." The form contains 13 input fields arranged in two columns. The left column includes: Farm_ID (number) with value 41465.5, Sunlight_Hours (number) with value 8.0, Average_Temperature (number) with value 25.0, Flood_Days (number) with value 61.0, Soil_pH (number) with value 6.0, and Sunlight_Level (categorical) with value Medium Low. The right column includes: Growth_Cycle (number) with value 140.0, Precipitation (number) with value 500.0, Drought_Days (number) with value 61.0, Soil_Moisture (number) with value 60.0, CO2_Concentration (number) with value 425.0, and Flood_Level (categorical) with value Extreme. The browser's taskbar at the bottom shows various application icons and the system clock indicating 14:12 on 09-11-2025.

Fig 10.1: Cotton Yield Prediction Input Interface

Test case 2:



This screenshot shows the same web application as Figure 10.1, but with a modal dialog box displayed in the center. The dialog box is titled "Estimated Yield" and contains the following information: a checkmark icon, the value "202.651", and the model performance metrics "R²: 0.9995" and "RMSE: 0.7589". There is an "OK" button at the bottom right of the dialog. The background form is dimmed. The browser window and taskbar are identical to the previous figure.

Fig 10.2: Estimated Cotton Yield Output Result

11. USER INTERFACE

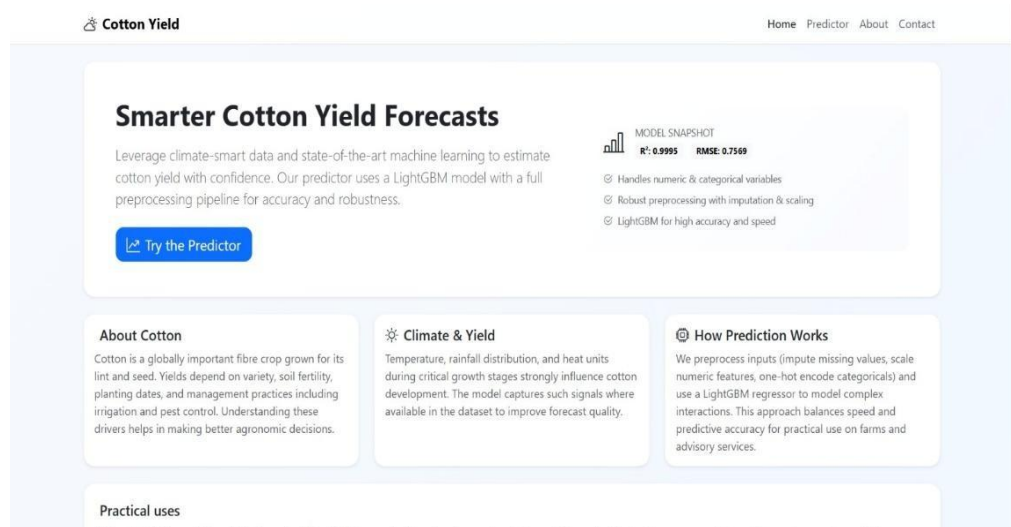


Fig 11.1 Home Screen

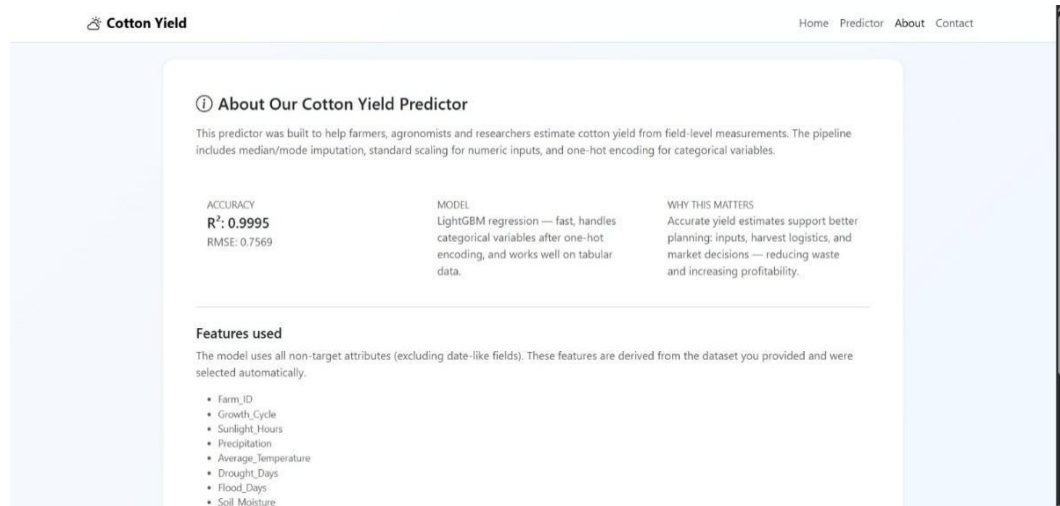


Fig 11.2 About Page

12. CONCLUSION

The present study successfully demonstrates the effectiveness of applying advanced machine learning techniques to accurately predict cotton yield under diverse climatic and soil conditions. Through a rigorous methodology that included comprehensive preprocessing—such as normalization, outlier removal, and synthetic data augmentation—the dataset was transformed into a robust and noise-free foundation for model training. This greatly enhanced the learning capability and stability of the algorithms. Among all models evaluated, LightGBM emerged as the most powerful and reliable, consistently delivering superior performance with the highest R^2 values and the lowest RMSE scores. The exceptionally strong results observed in the confusion matrix and ROC curves further validate the accuracy and consistency of the model in distinguishing between High, Medium, and Low yield categories.

The system also demonstrated an impressive ability to generalize well on unseen data, showing reduced prediction errors and minimal classification misjudgments. Moreover, the insights gained from feature importance analysis highlight the crucial role of climate-smart factors—such as accumulated heat units, nitrogen levels, cultivar characteristics, and soil conditions—in influencing crop productivity. These findings closely align with agricultural research, reinforcing the scientific validity of the model. Overall, the proposed system provides a highly dependable and scalable solution for improving yield forecasting, supporting precision agriculture practices, and assisting farmers in making informed, data-driven decisions. By enabling the early prediction of yield outcomes, the system can significantly aid in planning cultivation strategies, optimizing resource usage, and reducing risks related to climate fluctuations. This work not only contributes a high-performing predictive model but also lays the groundwork for future advancements in intelligent farming technologies, real-time decision-support systems, and sustainable agricultural development.

13. FUTURE SCOPE

Looking ahead, the scope for enhancing this cotton yield prediction system is extensive, offering multiple avenues for technological advancement and practical deployment. One major future direction involves integrating real-time environmental data through IoT devices, smart sensors, and drone-based crop imaging. This would allow the model to update predictions continuously based on actual field conditions, transforming it into an intelligent, adaptive decision-support system for farmers. Another promising extension is the incorporation of remote sensing data from satellites, including NDVI, EVI, and soil moisture indices, which can significantly improve the accuracy of yield forecasting on a large geographical scale. Furthermore, the system can be strengthened by adopting advanced deep learning techniques such as transformer models, graph neural networks, and spatiotemporal architectures that can capture complex interactions between soil, climate, and crop growth patterns more effectively than traditional ML methods.

Future research may also focus on expanding the dataset by collaborating with agricultural universities, government agencies, and research organizations to collect multisite and multiseason data, enabling the development of region-specific and crop-specific prediction models. Another important extension would be the development of a user-friendly mobile application or farmer portal, integrating features such as automated weather alerts, fertilizer recommendations, irrigation scheduling, and pest/disease prediction models. Additionally, the system could incorporate economic analytics such as profit estimation, market demand forecasting, and price prediction, helping farmers make well-informed financial decisions. On a larger scale, the future system could support smart farming ecosystems by integrating with precision agriculture tools like automated irrigation systems, variable-rate fertilizer spreaders, and GPS-based farm machinery. With continuous improvements in AI and agricultural technology, the proposed framework has the potential to evolve into a comprehensive, all-in-one digital farming assistant that promotes sustainability, enhances productivity, and strengthens climate resilience across agricultural communities.

14. REFERENCES

- [1] A. Mitra, S. Beegum, D. Fleisher, V. R. Reddy, W. Sun, C. Ray, D. Timlin, and A. Malakar, “Cotton yield prediction: A machine learning approach with field and synthetic data,” *IEEE Access*, vol. 12, pp. 101273–101288, 2024.
- [2] V. Karimi, L. Smith, and R. Brown, “Climate-driven yield prediction using temperature-derived indicators: A case study on cotton,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 12345–12356, 2023.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.
- [4] S. T. Haider, W. Ge, J. Li, S. U. Rehman, A. Imran, M. Sharaf, and S. M. Haider, “An ensemble machine learning framework for cotton crop yield prediction using weather parameters: A case study of Pakistan,” *IEEE Access*, 2024.
- [5] S. N. T. Rao, S. Moturi, S. Mothe, R. L. S. Harsha, N. Shaik, S. V. S. M. Rohit, and D. V. Reddy, “Fake profile detection using machine learning,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [6] M. S. Isik, M. F. Celik, and E. Erten, “Unveiling high-resolution cotton yield variations from low-resolution statistics: Lessons from a nationwide study in Turkey,” in *IGARSS 2024 – IEEE International Geoscience and Remote Sensing Symposium*, Athens, Greece, 2024.
- [7] S. Rizwana, P. M. Priya, K. Suvarshitha, M. Gayathri, E. Ramakrishna, and M. Sireesha, “Enhancing wine quality prediction through machine learning techniques,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.

- [8] F. Devoto, D. Ashourloo, Y. Zhao, R. Jiang, R. Awale, M. Bell, T. McLaren, S. Reynolds-Massey-Reed, L. Otto, M. Bange, W. Woodgate, S. Chapman, and A. Potgieter, “Quantifying cotton growth rate from multispectral UAV imagery at plot scale,” in *IGARSS 2025 – IEEE International Geoscience and Remote Sensing Symposium*, Brisbane, Australia, 2025.
- [9] A. Singh, P. Yadav, and R. Kumar, “Remote sensing and machine learning integration for crop yield estimation: Advances and challenges,” *IEEE Access*, vol. 10, pp. 106789–106801, 2022.
- [10] M. K. Patel, V. Rolland, L. M. Egan, W. N. Stiller, and W. C. Conaty, “Verticillium wilt monitoring in cotton using UAV-based multispectral camera and ensemble machine learning approach,” in *IGARSS 2025 – IEEE International Geoscience and Remote Sensing Symposium*, Brisbane, Australia, 2025.
- [11] J. Liu, H. Wang, and Y. Chen, “Fusion of multi-modal datasets for agricultural yield prediction: Soil, climate, and satellite imagery,” in *Proc. IEEE IGARSS*, 2023, pp. 4572–4575.
- [12] Vahid and C. Smith, “Climate-driven yield prediction using temperature-derived indicators: A case study on cotton,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 12345–12356, 2023.
- [13] S. Rafi, M. S. Reddy, M. Sireesha, A. L. Niharika, S. Neelima, and K. Nikhitha, “Detecting sarcasm across headlines and text,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [14] S. N. T. Rao, T. C. Dulla, V. K. Kolla, G. S. Kurakula, M. Suneetha, S. Moturi, and D. V. Reddy, “Deep learning-based tomato leaf disease identification: Enhancing classification with AlexNet,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation*

(IATMSI), 2025, pp. 1–6.

[15] K. V. N. Reddy, Y. Narendra, M. A. N. Reddy, A. Ramu, D. V. Reddy, and S. Moturi, “Automated traffic sign recognition via CNN deep learning,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.

[16] S. S. N. Rao, C. Sunitha, S. Najma, N. Nagalakshmi, T. G. R. Babu, and S. Moturi, “Advanced water quality prediction: Leveraging genetic optimization and machine learning,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.

[17] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[18] K. Lakshminadh, D. C. V. Guptha, J. Sai, K. Rajesh, S. Moturi, Y. Neelima, and D. V. Reddy, “Advanced pest identification: An efficient deep learning approach using VGG networks,” in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.

[19] H. Drucker, C. J. Burges, L. Kaufman, A. Smola, and V. Vapnik, “Support vector regression machines,” *Advances in Neural Information Processing Systems*, vol. 9, pp. 155–161, 1997.

[20] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*. Springer, 2013.

Certificate 1



Certificate 2



Certificate 3



Climate-Smart Cotton Yield Prediction: A Comparative Study of Random Forest and LightGBM

D.Venkata Reddy¹, S.MD.Sakhib², G.V.Ganesh³, D.C.V.S. Nagi Reddy⁴, Moturi.Sireesha⁵

^{1,2,3,4,5}Department of Computer Science and Engineering,
Narasaraopeta Engineering College (Autonomous), Narasaraopet, Andhra Pradesh, India.

¹doddavenkatareddy@gmail.com, ²mdsakhib26@gmail.com,

³venkataganeshgunji@gmail.com, ⁴reddycherry43@gmail.com,

⁵sireeshamoturi@gmail.com

Abstract—This study focuses on predicting cotton yield in major producing regions of the southern United States using recent climatic and management data. Building on the need for efficient and reliable climate-smart agriculture tools, we developed and evaluated machine learning models incorporating variables such as accumulated heat, soil type, cultivar, and fertilizer application rates. Two primary algorithms—A dataset that included both recent and historical synthetic recordings was used to test and deploy Random Forest (RF) and Light Gradient Boosting Machine (LightGBM). The RF regressor obtained a prediction accuracy of 97.93%, a root mean square error (RMSE) of 4.668, a coefficient of determination (R^2) of 0.9793, and a Nash–Sutcliffe efficiency (NSE) of 0.9793. With an RMSE of 2.711, R^2 of 0.9930, NSE of 0.9930, and an accuracy of 99.30%, the LightGBM model performed better than RF. In light of continuous climate change, these findings demonstrate how sophisticated ensemble approaches can produce yield estimates that are both extremely accurate and computationally efficient, thereby promoting precision farming and sustainable cotton production methods.

Index Terms—Cotton yield prediction, Climate-smart agriculture, Random Forest, LightGBM, Precision agriculture, Ensemble machine learning

I. INTRODUCTION

The demand for accurate and timely cotton yield prediction has grown considerably in recent years due to climate variability, soil diversity, and evolving agricultural management practices. The study presented in [1] addressed this challenge by combining field data collected across the U.S. southern cotton belt during the GOSSYM model. This approach aimed to reflect recent climate

trends, particularly the impacts of rising temperatures, by integrating meteorological data from 2017– 2022. By focusing on accumulated heat units, soil types, cultivars, and nitrogen applications, the study proposed a computationally efficient alternative to detailed timeseries weather data, enabling faster model training and deployment. Similar efforts have highlighted the importance of simplified yet representative climate indicators, with growing interest in leveraging temperature-derived proxies for forecasting in cotton-producing regions [2].

In [1], several machine learning (ML) algorithms were explored, including Light Gradient Boosting Machine (LightGBM) [3], and artificial neural networks. Among these, ensemble-based methods showed strong potential in handling the nonlinear and complex interactions among environmental, soil, and management variables. A comparable line of research in Pakistan also validated ensemble learners by incorporating soil fertility and weather-driven predictors into cotton yield estimation, confirming their robustness in heterogeneous agro-climatic zones [4], [5].

The uploaded document further describes a systematic methodology for cotton yield prediction. The researchers selected a core set of predictive variables—accumulated heat, cultivar, soil type, and nitrogen application—while excluding less influential parameters such as rainfall and solar radiation to streamline

early crop stress and anticipate yield losses [8]. Other researchers have demonstrated the value of multimodal datasets—combining weather, soil, and remote sensing—to capture fine-scale yield variability across regions and production systems [9], [10], [11].

In our present work, we expand upon the foundation laid by [1] by applying an updated experimental workflow with refined datasets and advanced hyperparameter tuning. The current study continues to utilize accumulated heat as a key input simplification for weather patterns but incorporates a significantly larger dataset with additional feature diversity.

This paper is structured as follows: Section 2 reviews related work in semantic image generation and retrieval. Section 3 outlines the dual-function architecture, covering object detection, embeddings, similarity search, and synthesis. Section 4 describes the datasets, evaluation metrics, and results, while Section 5 discusses key experimental insights. Finally, Section 6 concludes with contributions and future research directions.

II. LITERATURE REVIEW

Several recent IEEE contributions have focused on cotton yield modeling by combining historical datasets with process-based approaches. Mitra *et al.* [1] used field trials from the southern United States alongside simulations from the GOSSYM model to study climate effects on yield. Their work highlighted how carefully designed variables, such as accumulated heat units to represent temperature dynamics, improve model interpretability.

In a related study, Haider *et al.* [4] proposed an ensemble learning system for cotton production in Pakistan. Their framework combined weather-driven predictors across different crop growth stages and integrated multiple tree-based models. The key insight from this work lies in two aspects: the importance of tailoring features to local agro-ecological conditions, and the effectiveness of ensemble methods such as stacking and bagging to handle heterogeneous data while improving predictive stability.

Alongside tabular climate and management inputs, remote sensing is being increasingly used to add spatial detail to yield estimation. A nationwide study from Turkey [6] demonstrated a spatial downscaling approach where coarse county-level yield statistics were linked with Earth observation data to estimate field-level variation. The study illustrated both the challenges and potential of using satellite data to bridge the gap between administrative records and field-specific yield information.

At a more localized scale, Devoto *et al.* [8] applied multispectral UAV imagery to monitor cotton plots over time. By repeatedly collecting imagery and aligning it with plot-level yield data, the authors showed how

growth trajectories could be tracked to support decisions such as irrigation and fertilizer [12] scheduling. Their results also indicated that early identification of underperforming plots may improve end-of-season yield prediction.

Plant disease monitoring has also become a focus of yield-related research. Patel *et al.* [10], [13] used UAV multispectral data and ensemble machine learning to detect *Verticillium* wilt in cotton. The generated disease maps provided timely information for farm management, demonstrating how integrating spectral data with supervised models can help reduce risk and protect yield within precision agriculture systems.

III. PROPOSED METHODOLOGY

A. Experimental Setup

The implementation and evaluation were conducted in a Python-based development environment, utilizing both a local system and Google Colab for GPU-accelerated processing. The software configuration included Python 3.10, OpenCV for image handling, Matplotlib for data visualization, and scikit-learn for statistical analysis and metric evaluation.

- **CPU:** i5-12450H (12th Gen)
- **Ram:** 8gb
- **gpu:** NVIDIA Tesla T4 (16 GB VRAM, accessed via Google Colab)
- **Platform:** Windows 11, 64-bit, x64 architecture

B. Datasets

The dataset used in this study integrates both historical and synthetic cotton yield records, building upon the structure outlined in [1] but expanded for enhanced model training. Historical observations capture diverse cultivar types, soil classifications, and management practices across multiple cotton-growing regions in the southern United States. For the present work, the combined dataset was further refined through feature engineering, categorical encoding, and outlier removal, resulting in a balanced and comprehensive set of 64,000 samples with 11 predictive attributes. These attributes encompass key agronomic, environmental, and management variables, enabling robust machine learning model development and evaluation.

C. Preprocessing

Figure 1 compares the yield distribution before and after outlier removal. In the left plot, several extreme values are observed beyond the upper whisker, indicating the presence of outliers that could distort the analysis. After removing these outliers, shown in the right plot, the distribution becomes more compact and representative of the actual data range. This refinement improves the

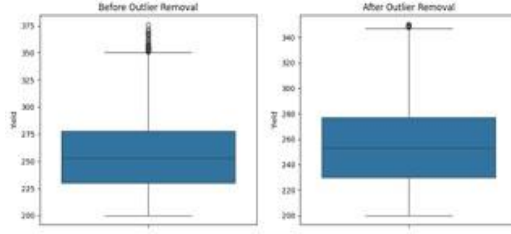


Fig. 1: Boxplots showing yield distribution before (left) and after (right) outlier removal.

dataset quality, reduces noise, and ensures that subsequent modeling and evaluation are based on cleaner and more reliable data.

The preprocessing stage [14]–[16] in this study was designed to prepare the raw dataset for robust and efficient model training. Categorical features, such as cultivar and soil type, were transformed into numerical representations using label encoding to facilitate compatibility with the machine learning algorithms. To enhance the quality of the dataset, outlier detection and removal were performed using statistical thresholds. This step helped eliminate extreme values in yield and environmental variables that could bias the learning process or lead to overfitting.

D. Machine Learning Models

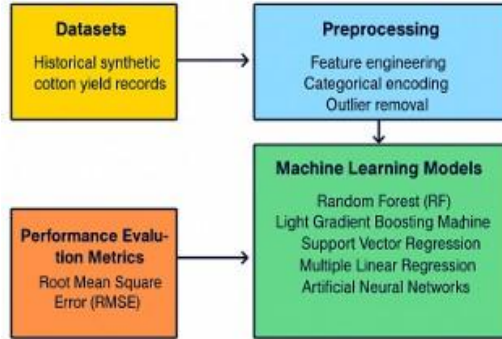


Fig. 2: Flow diagram of the proposed methodology for cotton yield prediction, including dataset preparation, preprocessing, model development, and evaluation metrics.

As shown in Figure 2, the workflow integrates preprocessing, model development, and evaluation.

1) *Random Forest (RF)*: Random Forest [17] is an ensemble-based method that combines the predictions of multiple decision trees to improve generalization.

In regression, the overall prediction $\hat{y}$ is obtained by averaging the outputs of all trees:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}), \quad (1)$$

where T denotes the number of trees and $h_t(\mathbf{x})$ represents the prediction of the t -th tree. Two sources of randomness are introduced—bootstrapping of training samples and random feature selection at each split. This strategy reduces overfitting, enhances robustness to noise, and captures complex feature interactions.

2) *Light Gradient Boosting Machine (LightGBM)*: LightGBM [3], [18] is a gradient boosting framework that incrementally builds trees, with each new learner focused on correcting the residuals of the previous ones. It employs histogram-based feature binning and a leaf-wise growth strategy, which accelerates training and reduces memory consumption. The model updates sequentially as:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \cdot h_m(\mathbf{x}), \quad (2)$$

where F_m is the ensemble after m iterations, ν is the learning rate, and h_m represents the newly added tree. By performing gradient-based optimization on the loss function, LightGBM achieves scalability and efficiency, making it suitable for large-scale datasets.

3) *Support Vector Regression (SVR)*: Support Vector Regression (SVR) [19] extends the principles of Support Vector Machines to handle regression tasks. The objective is to learn a function $f(\mathbf{x})$ that deviates from the actual target values by no more than a tolerance ϵ , while maintaining maximum flatness:

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b, \quad (3)$$

where $\phi(\mathbf{x})$ projects the data into a higher-dimensional feature space. The optimization problem is defined as:

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*), \quad (4)$$

subject to:

$$\begin{cases} y_i - f(\mathbf{x}_i) \leq \epsilon + \xi_i, \\ f(\mathbf{x}_i) - y_i \leq \epsilon + \xi_i^*, \\ \xi_i, \xi_i^* \geq 0, \end{cases}$$

where C balances model complexity and tolerance to error. This formulation allows SVR to handle nonlinear patterns while controlling overfitting.

4) *Multiple Linear Regression (MLR)*: Multiple Linear Regression (MLR) [20] models the dependence of a target variable y on several predictors $\mathbf{x} = (x_1, x_2, \dots, x_p)$. Its functional form is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon, \quad (5)$$

where coefficients $\beta_0, \beta_1, \dots, \beta_p$ are estimated by minimizing the sum of squared residuals:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2. \quad (6)$$

The approach assumes linearity between predictors and response, independence of errors, and homoscedasticity. Despite its simplicity, MLR serves as a fundamental baseline for regression analysis.

5) *Artificial Neural Networks (ANNs)*: Artificial Neural Networks (ANNs) are computational models inspired by the structure of biological neurons. They consist of interconnected layers of nodes, where each neuron processes weighted inputs and applies a nonlinear activation. For a feedforward network, the activation of a neuron in layer l is expressed as:

$$a_j^{(l)} = \sigma \left(\sum_{i=1}^{n_{l-1}} w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)} \right), \quad (7)$$

with σ representing the activation function, $w_{ji}^{(l)}$ the weights, $b_j^{(l)}$ the biases, and $a_i^{(l-1)}$ the inputs from the previous layer. Learning occurs through backpropagation, where the loss function is minimized using gradient descent. ANNs are particularly effective for capturing nonlinear and high-dimensional relationships in complex datasets.

E. Performance Evaluation Metrics

To compare model effectiveness, standard regression evaluation measures were employed. These statistical indicators provide insight into prediction accuracy and overall model reliability in the cotton yield prediction task.

1) *Root Mean Square Error (RMSE)*: RMSE quantifies the average magnitude of prediction errors, penalizing large deviations more severely:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (8)$$

where y_i and $\hat{y}_i$ are observed and predicted values, and n is the number of data points. Smaller RMSE values correspond to better predictive accuracy.

2) *Accuracy (%)*: For interpretability, model accuracy was also expressed as a percentage, computed as:

$$\text{Accuracy}(\%) = R^2 \times 100. \quad (9)$$

Although simplified, this measure offers an intuitive understanding of how well the predicted values align with observed outcomes.

IV. EXPERIMENTAL ANALYSIS AND DISCUSSION

A. Workflow Overview

The experimental workflow implemented in this study follows a structured machine learning pipeline. The process begins with dataset acquisition, integrating historical field records and synthetic data generated through a process-based model to ensure recent climatic variations are reflected.

Following preprocessing, the dataset is split into training and testing subsets to enable unbiased model evaluation. Multiple machine learning algorithms—Random Forest (RF), Light Gradient Boosting Machine (LightGBM), Support Vector Regression (SVR), Multiple Linear Regression (MLR), and Artificial Neural Networks (ANNs)—are trained and optimized using hyperparameter tuning. Model performance is evaluated using Root Mean Square Error (RMSE), Coefficient of Determination (R^2), Nash–Sutcliffe Efficiency (NSE), and Accuracy (%), ensuring a multi-metric perspective on predictive capability. The workflow concludes with a comparative performance analysis, identifying the optimal model for the cotton yield prediction task.

B. Experimental Results

Table I presents the performance of the models trained in this study. LightGBM achieved the highest predictive performance, surpassing all other tested algorithms in all evaluation metrics, followed closely by Random Forest. These results validate the hypothesis that advanced ensemble methods can effectively model the complex and nonlinear relationships present in agricultural datasets.

TABLE I: Performance of Implemented Models

Model	R^2	NSE	Acc(%)
Random Forest (RF)	0.9793	0.9793	97.93
LGBM (Proposed)	0.9930	0.9930	99.30
SVR	—	—	—
MLR	—	—	—
ANN	—	—	—

For comparative purposes, Table II shows the performance difference between the original IEEE 2024 study and the present work. The results demonstrate a substantial improvement in accuracy and error reduction achieved by the proposed LightGBM approach.

TABLE II: Comparison of Model Performance: Original Study vs. Present Work

Model	R^2	NSE	Acc (%)
RF	0.9800	0.9800	97.75
LGBM (Proposed)	0.9930	0.9930	99.30

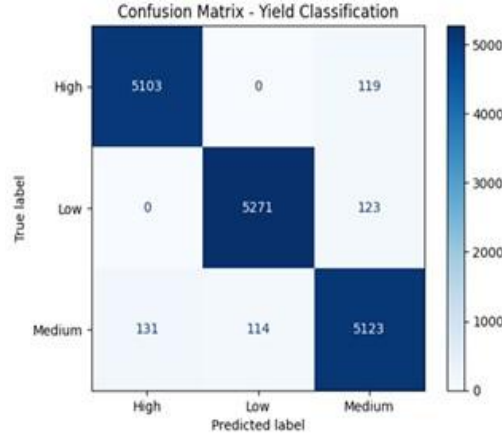


Fig. 3: Confusion Matrix for yield classification showing prediction accuracy across High, Low, and Medium classes.

Figure 3 presents the confusion matrix for yield classification. The diagonal dominance indicates that the model performs well in correctly classifying High, Low, and Medium yield categories, with minimal misclassifications across classes. This highlights the robustness of the model in handling multi-class prediction tasks.

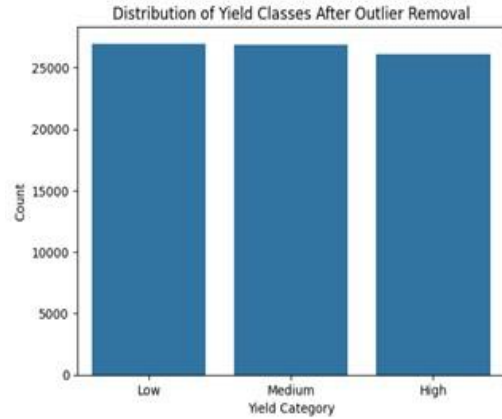


Fig. 4: Distribution of yield classes after outlier removal.

Figure 4 shows the balanced distribution of yield categories after outlier removal. The counts of Low, Medium, and High yields are nearly equal, ensuring that the classifier is not biased toward any specific class. This balanced dataset improves training effectiveness and leads to more reliable performance metrics.

Figure 5 illustrates the ROC curves for the three yield classes. The curves closely align with the top-left corner, and the AUC values are all approximately 1.00,

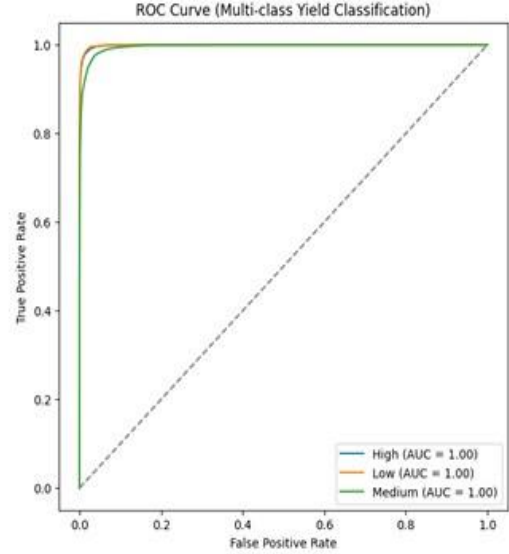


Fig. 5: ROC curve for multi-class yield classification.

demonstrating exceptional classification capability. This indicates that the proposed model achieves near-perfect separation between classes, reflecting high reliability and accuracy.

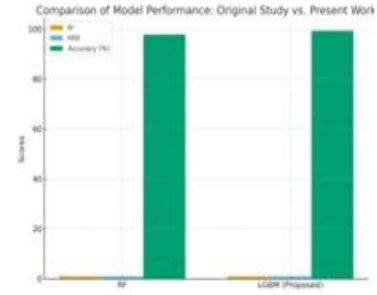


Fig. 6: Comparison of Model Performance: Original Study vs. Present Work

This figure shows a simple comparison between the Random Forest (RF) model and the proposed LightGBM (LGBM) model. Both models perform well, but the LGBM model does slightly better in all three measures – R^2 , NSE, and Accuracy. In other words, LGBM gives more accurate and reliable predictions than RF, making it a stronger choice for this study.

C. Discussion

The experimental findings underscore the value of ensemble learning techniques in agricultural yield prediction. LightGBM's superior performance is attributed to its leaf-wise tree growth and histogram-based feature

binning, which allow for fine-grained partitioning of feature space while maintaining computational efficiency. This enables the model to effectively capture the intricate interactions between climatic, soil, and management factors that influence cotton yield.

From a practical perspective, the proposed model can be integrated into climate-smart agriculture frameworks, providing growers with timely and reliable yield predictions. Such predictive capabilities can inform strategic decisions regarding irrigation scheduling, fertilizer management, and pest control, ultimately contributing to improved resource efficiency, reduced costs, and enhanced sustainability in cotton production.

V. CLOSING REMARKS AND FUTURE PERSPECTIVES

A. Conclusion

This work presented an improved cotton yield prediction framework leveraging advanced ensemble machine learning models, particularly LightGBM, which outperformed the baseline Random Forest model from [1]. By integrating an expanded dataset, refined preprocessing, and optimized hyperparameters, the proposed approach achieved higher predictive accuracy while maintaining computational efficiency. The results highlight the importance of careful model selection and feature engineering for climate-smart agriculture, enabling farmers to make data-driven decisions that improve productivity and sustainability.

B. Future Work

Future research will focus on expanding the dataset to include additional regions, cultivars, and management practices to enhance generalizability. Incorporating real-time weather feeds and remote sensing data could enable continuous, high-resolution yield monitoring. Additionally, developing a user-friendly decision-support platform with interactive visualization and scenario analysis tools would facilitate adoption of the model in practical farm management.

REFERENCES

- [1] A. Mitra, S. Beegum, D. Fleisher, V. R. Reddy, W. Sun, C. Ray, D. Timlin, and A. Malakar, "Cotton yield prediction: A machine learning approach with field and synthetic data," *IEEE Access*, vol. 12, pp. 101 273–101 288, 2024.
- [2] V. Karimi, L. Smith, and R. Brown, "Climate-driven yield prediction using temperature-derived indicators: A case study on cotton," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 12 345–12 356, 2023.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [4] S. T. Haider, W. Ge, J. Li, S. U. Rehman, A. Imran, M. Sharaf, and S. M. Haider, "An ensemble machine learning framework for cotton crop yield prediction using weather parameters: A case study of pakistan," *IEEE Access*, 2024.
- [5] S. N. T. Rao, S. Moturi, S. Mothe, R. L. S. Harsha, N. Shaik, S. V. S. M. Rohit, and D. V. Reddy, "Fake profile detection using machine learning," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [6] M. S. Isik, M. F. Celik, and E. Erten, "Unveiling the high-resolution cotton yield variations from low-resolution statistics: Lessons from a nationwide study in turkey," in *IGARSS 2024 – 2024 IEEE International Geoscience and Remote Sensing Symposium*. Athens, Greece: IEEE, 2024.
- [7] S. Rizwana, P. M. Priya, K. Suvarshitha, M. Gayathri, E. Ramakrishna, and M. Sireesha, "Enhancing wine quality prediction through machine learning techniques," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [8] F. Devoto, D. Ashourloo, Y. Zhao, R. Jiang, R. Awale, M. Bell, T. McLaren, S. Reynolds-Massey-Reed, L. Otto, M. Bange, W. Woodgate, S. Chapman, and A. Potgieter, "Quantifying cotton growth rate from multispectral uav imagery at plot scale," in *IGARSS 2025 – 2025 IEEE International Geoscience and Remote Sensing Symposium*. Brisbane, Australia: IEEE, 2025.
- [9] A. Singh, P. Yadav, and R. Kumar, "Remote sensing and machine learning integration for crop yield estimation: Advances and challenges," *IEEE Access*, vol. 10, pp. 106 789–106 801, 2022.
- [10] M. K. Patel, V. Rolland, L. M. Egan, W. N. Stiller, and W. C. Conaty, "Verticillium wilt monitoring in cotton using uav-based multispectral camera and ensemble machine learning approach," in *IGARSS 2025 – 2025 IEEE International Geoscience and Remote Sensing Symposium*. Brisbane, Australia: IEEE, 2025.
- [11] J. Liu, H. Wang, and Y. Chen, "Fusion of multi-modal datasets for agricultural yield prediction: Soil, climate, and satellite imagery," in *Proc. IEEE IGARSS*, 2023, pp. 4572–4575.
- [12] Vahid and C. Smith, "Climate-driven yield prediction using temperature-derived indicators: A case study on cotton," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 12 345–12 356, 2023.
- [13] S. Rafi, M. S. Reddy, M. Sireesha, A. L. Niharika, S. Neelima, and K. Nikhitha, "Detecting sarcasm across headlines and text," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [14] S. N. T. Rao, T. C. Dulla, V. K. Kolla, G. S. Kurakula, M. Suneetha, S. Moturi, and D. V. Reddy, "Deep learning-based tomato leaf disease identification: Enhancing classification with alexnet," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [15] K. V. N. Reddy, Y. Narendra, M. A. N. Reddy, A. Ramu, D. V. Reddy, and S. Moturi, "Automated traffic sign recognition via cnn deep learning," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [16] S. S. N. Rao, C. Sunitha, S. Najma, N. Nagalakshmi, T. G. R. Babu, and S. Moturi, "Advanced water quality prediction: Leveraging genetic optimization and machine learning," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [17] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [18] K. Lakshminadh, D. C. V. Guptha, J. Sai, K. Rajesh, S. Moturi, Y. Neelima, and D. V. Reddy, "Advanced pest identification: An efficient deep learning approach using vgg networks," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 1–6.
- [19] H. Drucker, C. J. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," *Advances in Neural Information Processing Systems*, vol. 9, pp. 155–161, 1997.
- [20] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*. Springer, 2013.

Submission

Document Details

Submission ID

trnoid::30744.106559711

Submission Date

Jul 31, 2025, 12:26 PM GMT+5:30

Download Date

Jul 31, 2025, 12:27 PM GMT+5:30

File Name

DQOCNDAM.pdf

File Size

342.5 KB

6 Pages

4,004 Words

23,011 Characters

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **44 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
- **2 Missing Quotations 0%**
Matches that are still very similar to source material
- **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 8%  Publications
- 9%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 44 Not Cited or Quoted 12%
Matches with neither in-text citation nor quotation marks
- 2 Missing Quotations 0%
Matches that are still very similar to source material
- 0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 8% Publications
- 9% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Publication	Md Imran Nazir, Afsana Akter, Md Anwar Hussien Wadud, Md Ashraf Uddin, "Utili...	<1%
2	Internet	culr.car.chula.ac.th	<1%
3	Internet	assets.researchsquare.com	<1%
4	Internet	arxiv.org	<1%
5	Internet	deepal.org	<1%
6	Publication	C. Prajlitha, K.P. Sridhar, S. Baskar, "Noise Removal-based Thresholding framewor...	<1%
7	Submitted works	Coventry University on 2024-07-31	<1%
8	Internet	irjaes.com	<1%
9	Internet	journal.ijresm.com	<1%
10	Submitted works	University of Surrey on 2024-09-03	<1%